Array 28 (2025) 100597

Contents lists available at ScienceDirect

Array

journal homepage: www.sciencedirect.com/journal/array

Efficient management of computational resources in the IoT FOG layer. Automation and load balancing using Simple FOG Management Protocol

Lucia Arnau Muñoz *, José Vicente Berná Martínez, David Saavedra Pastor, Iren Lorenzo Fonseca

University of Alicante, Carretera San Vicente del Raspeig s/n, 03690, San Vicente del Raspeig, Spain

ARTICLE INFO

Keywords:
Dynamic FOG architecture
Simple FOG Management Protocol
Resource allocation
Energy efficiency in IoT

ABSTRACT

FOG computing is an essential paradigm for Internet of Things (IoT) environments, as it enables the decentralisation of data processing to make better use of resources. The rapid proliferation of IoT devices has created new challenges for this middle layer, especially in terms of reconfiguration and dynamic scalability, processes that have traditionally been managed manually. This study presents an architecture for the FOG layer that implements dynamic control and scaling automation through an advanced control layer composed of a coordinator that calculates the optimal distribution of computational load and a balancer that manages traffic routing. The central component of the proposal is the Simple FOG Management Protocol (SFMP), designed specifically to facilitate automated resource management, ensuring efficient allocation of computational capacity, fair load balancing, and robust fault tolerance mechanisms. Experimental validation in several complex IoT environments at the University of Alicante demonstrated a substantial optimisation in resource utilisation with a reduction in energy consumption of between 40 % and 65 %. All this confirms the flexibility and adaptability of the system to dynamic variations in computational demand in large-scale IoT implementations and its benefits over traditional proposals.

1. Introduction

The use of IoT devices to sensor and act on environments has led to many benefits in different contexts such as agriculture [1], healthcare [2], industrial manufacturing [3], homes [4] and even educational environments [5]. Through IoT networks, it is possible to know the real-time status of multiple sensorised values of the context and to realise efficient energy management, increased safety for people, better resource utilisation, automate operations, improve privacy, counteract the effects of climate change, improve crop production, detect leaks and failures, and many other benefits. Such hyper-connected environments, with a range of heterogeneous technologies, using multiple protocols and platforms, which both feed and are fed by a wide variety of technologies, require a large number of diverse infrastructures for their execution. In the past, we used to find large computing centres where all the execution capacity of the computer systems resided, and little by little, due to the hyper-decentralisation of these computing systems [6], we find ourselves with very diverse architectures, with multiple paradigms and hardware located in different locations, where different layers specialise in certain functionalities, and in which it is very difficult to ensure efficiency [7].

This new scenario reveals a new issue, which is the efficient use of computing resources in IoT environments [8]. In a context with a large number of devices, a large amount of computational resources is required to collect, filter, clean, match, enrich and store the data generated from IoT devices. Tasks such as data analysis to generate an informed view of hidden patterns, missing data imputation and data aggregation, outlier detection, data fusion, feature extraction or noise removal must be addressed [9]. For these tasks, models have been proposed that structure the functionalities in several layers. The 3-layer models [10] are usually close to the physical scheme in which the system is divided, the perception, the communication network and the application. Later proposals extend the layers to 5 [11], adding a processing layer before the application layer and a business layer after, because the sophistication of IoT-based services is more complex, and it is necessary to add more and more diverse processing. But even in more complex

* Corresponding author.
E-mail addresses: lucia.arnau@ua.es (L. Arnau Muñoz), jvberna@ua.es (J.V. Berná Martínez), david.saavedra@ua.es (D. Saavedra Pastor), iren.fonseca@ua.es (I. Lorenzo Fonseca).

https://doi.org/10.1016/j.array.2025.100597
Received 13 July 2025; Received in revised form 28 October 2025; Accepted 21 November 2025
Available online 21 November 2025
2590-0056/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

systems, it is possible to talk about 7 layers, adding an intermediate analysis and transformation layer and a data abstraction layer [12]. These layers established from a functional perspective can be grouped into more abstract levels dependent on the physical location of the computing capabilities, known as EDGE, FOG and CLOUD. EDGE being a layer located close to the physical IoT devices, CLOUD is the layer closer to the application and services in enterprise infrastructures or large servers, and the FOG layer is an intermediate infrastructure, strategically placed to alleviate CLOUD computing and solve problems that cannot be solved by the limited capabilities of the EDGE. In this variety of proposals, FOG Computing is presented as a popular technology that tries to meet the needs of applications based on IoT devices, being everywhere and at any time [13].

However, although this situation allows solving the resource needs by providing each layer with the necessary infrastructures, it reveals a new problem, infrastructures are prone to inefficiency, since each layer must be provided with resources, which may be under-utilised at one level, while at other levels they may be saturated, and vice versa. This is why our proposal is based on a structured model, with a FOG layer heavily endowed with resources, but resources that can be used or not, depending on the computational needs of the system. The proposal structures the FOG layer in a series of elements for monitoring and controlling computing needs, which allow the system to remain stable. To do this, it uses an internal division of the FOG layer into levels, and the location of elements that dynamically scale resources, using the minimum resources necessary to ensure the processing required by the IoT. This FOG layer is able to automate and control the scaling of resources, thus generating a dynamic layer specially designed to be efficient in the use of infrastructures.

In our previous work [14], we presented the fundamental elements of a dynamic FOG architecture and the Simple FOG Management Protocol (SFMP) essential for automated resource management in IoT infrastructures. That initial proposal demonstrated the feasibility of dynamic scaling and load balancing in a single implementation on a university campus. However, several critical issues remained to be resolved: its long-term stability and fault tolerance under sustained production loads, its performance compared to traditional resource allocation techniques, and the actual overhead introduced by the management protocol under real-world conditions. This paper addresses these issues by introducing new replacement elements in the architecture and evolution of the SFMP to support fault tolerance. The article also presents the comprehensive validation tests of this new version, such as: deployment of the architecture on three geographically separate campuses of the University of Alicante; a 30-day continuous operation study to verify the stability and sustained efficiency of the system under real production conditions; comparative analyses with traditional resource allocation approaches to demonstrate the specific advantages of our proposal for IoT environments; and finally, an evaluation of the SFMP protocol overhead and fault recovery capabilities in operational environments that manage millions of messages per hour.

The rest of the article is organised in the following chapters. Section 2 provides a related work analysis of other proposals that pursue similar objectives to ours; Section 3 provides a description of the current situation of the campus, together with the problems that this presents; Section 4 describes the design of the FOG architecture that will host the solution; Section 5 describes the new components in the architecture, together with the existing components and their functionalities; Section 6 explains the new communication protocol developed for architecture management and error control; Section 7 describes the implementation of the architecture on the campus of the University of Alicante, specifying the tests carried out; Section 8 analyses the impact of the new protocol on the architecture's communications; and finally, Section 9 draws the main conclusions of the article and future lines of work.

2. Related works

FOG architectures represent a growing solution to solutions based solely on Cloud computing, mainly focused on addressing problems based on latency and bandwidth limitations that arise with the immense increase of IoT devices that are implemented every day in our society [15]. Some of the solutions used to obtain more efficient FOG systems are, for example, the intelligent distribution of workloads among the elements of the infrastructure, or the implementation of algorithms for resource management [16], in order to maximise the performance of the infrastructure, reduce the associated costs, and ensure the correct operation of the entire system [17]. Among the solutions mentioned is the importance of resource allocation. Infrastructure resources must always be secured and allocated efficiently, since when this requirement is met, it is often not necessary to expand the infrastructure resources, since those available are managed efficiently and a cost that could be unnecessary simply due to misuse of resources is avoided.

Providing the infrastructure nodes with sufficient computational resources [18] requires a correct and controlled allocation of resources, depending on the incoming workload originating from the IoT device layer. A distinction must be made between the concept of resource provisioning and resource allocation. An infrastructure may have sufficient resources, provisioning, but they may not be being used in an efficient way to optimise their use to the maximum, allocation [19]. Provisioning and allocation are two sides of the same coin, as a FOG node needs to provision resources in order to allocate them to IoT requests.

At present, there are three main areas with respect to the provisioning and allocation of resources:

2.1. Prior provisioning and prompt allocation (PPPA)

In this approach, resources are reserved in advance based on predictions of estimated IoT traffic [20]. These predictions use models that analyse historical data and trends, allowing estimation of future resource demand. The main advantage of this strategy is to ensure that resources are available at FOG nodes when IoT requests arrive, ensuring immediate and low-latency processing. This type of solution is useful in scenarios where continuous operation is required, with strictly measured response times, such as remote healthcare systems, autonomous vehicles or smart city infrastructures.

However, this strategy has important limitations, the most notable being that it relies heavily on the accuracy of the predictions, in case they are inaccurate, two main problems can occur: the first, over-provisioning, where more resources are allocated than necessary, leading to wasted capacity; the second, on the contrary, under-provisioning, which leaves applications without sufficient resources to guarantee their performance. In addition, the calculations needed to predict traffic and reserve resources in advance may require high computational cost in order to have accurate estimates.

An example of the use of such solutions could be in a smart city network, where this approach could reserve resources at FOG nodes located near critical intersections during peak traffic hours. This ensures that traffic sensor data is processed without interruption to optimise traffic light synchronisation and minimise congestion. However, during off-peak hours, reserved resources may be underutilised, resulting in a waste of available resources. Therefore, accurate prediction of IoT traffic is essential, which can be considered a disadvantage in many different application scenarios.

2.2. On-demand provisioning and prompt allocation (ODPPA)

Unlike the previous approach, in this case resources are not reserved in advance [21]. IoT requests are dynamically processed upon arrival and assigned to the nearest available FOG node, without explicitly considering factors such as processing cost or resource availability. This

approach is notable for its flexibility, as it adapts well to dynamic environments where the workload is constantly varying. It is especially useful in applications where it is not possible to accurately forecast demand, such as in IoT networks.

However, this strategy has significant disadvantages. Since no prior optimisation is performed, it can lead to inefficient use of resources. Nearby nodes could quickly become overloaded while others remain underutilised, causing possible congestion and increasing processing latency. Within this approach, there are different processing strategies at FOG nodes. One of them is concurrent processing [22], where several IoT applications are processed in parallel on the same node, regardless of the current resource capacity. While this allows handling large volumes of data, it can result in high latency when node resources are insufficient. Another common strategy is First-Come, First-Served (FCFS) [23], which processes IoT requests in the order in which they arrive at the FOG node. Although this strategy is simple to implement, it does not take into account the priority of requests and quality of service (QoS) requirements. A third strategy is delay-priority [24], where requests are processed based on their latency or QoS requirements. For example, a request with a critical response time will be prioritised over other less urgent requests, which improves performance for critical applications.

A practical example of this approach could be a network of surveillance cameras connected to a security system in a city [25]. These cameras send real-time data streams to FOG nodes for analysis, such as suspicious movement detection or facial recognition. When activity is detected in an area with several cameras operating simultaneously, requests are processed directly at the nearest node. This approach allows data to be analysed quickly, but if a node receives too many requests at the same time, it could become overloaded, leading to delays or even loss of critical data. For this reason, it is necessary to monitor the load of the nodes and consider implementing strategies capable of balancing the workload of the system.

2.3. On-demand provisioning and small batch allocation (ODPSBA)

This approach combines the flexibility of on-demand provisioning with a more optimised approach to resource allocation [26]. Instead of processing each IoT request individually, the FOG resource manager aggregates a small batch of requests before allocating them to available nodes [27]. This grouping allows finding a more efficient allocation that optimises some global metric, such as latency, energy consumption or processing cost. Once a node is assigned to a request, all future requests for the same application are processed on that node, which improves long-term efficiency.

The main benefit of this approach is its ability to reduce costs and improve resource utilisation by processing multiple requests together. This is especially useful in static IoT applications, where requests have consistent patterns and do not require immediate responses. However, the accumulation of requests introduces a small initial latency, which can be a drawback for applications that need real-time processing. Also, in dynamic environments where demands are unpredictable, this approach may be less efficient compared to fast allocation.

A practical example could be an environmental monitoring system that collects data from temperature, humidity and air quality sensors in a nature park. Instead of processing each piece of data individually, the system aggregates requests from several sensors and assigns them to a FOG node for processing. This optimises energy consumption and ensures that resources are used efficiently, although it introduces a small delay in response.

2.4. Other ad-hoc proposals

On a more compilation basis, previous work has explained that some of the resource allocation schemes [28,29] assume that each node has a single task to process, while other work [30] proposes that different tasks can exist simultaneously in each node, processing more than one element at a time, when the infrastructure does not have the necessary resources to process the tasks separately. In both cases, each task is assigned to a single processing node, without providing the necessary flexibility required by these IoT systems.

In more advanced works [31], the idea of processing each message from IoT devices separately has been abandoned in favour of a strategy based on task bundling, i.e., concentrating all devices processing the same type of service to the same resource. However, with the constant evolution of sensing systems, these messages originating from the EDGE layer are becoming more and more sophisticated, being bundles composed of multiple tasks [32]. Each packet is composed of different types of sensors and actuators, resulting in processing that is too demanding for a single resource to handle, so more current solutions have concluded that the processing of incoming data from IoT devices needs to be split between one or more nodes.

The problem with much of this work is that it assumes static data traffic [33,34], with a completely reliable FOG infrastructure. In practice, many IoT devices are dynamic in terms of the frequency and flow of messages they send, such as sensors for presence monitoring, or counting people in a room, which can generate peaks of increased traffic coming into the infrastructure to be processed. Just as unexpectedly large volumes of data can affect the correct functioning of the system, so can infrastructure failures themselves, leading to the processing of incorrect or anomalous data, and the possibility of not being able to maintain the availability of the processing chain at all times [34].

In the event of a node failure, new incoming messages must be reallocated to a new functioning node, and one method of ensuring this fault tolerance is to add nodes to the infrastructure [35] when there is a predictable increase in incoming messages from IoT devices. However, this scaling of resources in a defined manner does not result in a long-term scalable solution either. One way to avoid this fixed increase in resources is to be aware of the resources available to the system at the outset [35], which facilitates decision-making when processing incoming messages to the FOG layer, but this is not an effective solution without thorough control of resource allocation in a fast and controlled manner.

In the end, computing at the FOG layer offers an alternative to processing at the EDGE layer and avoids unnecessarily saturating the CLOUD. The main problem encountered in this review of existing solutions is that none of them provide the requirements for the control of infrastructure resources. These requirements are: the flexibility to move processes from one processor node to another; and to allow for scalability in an efficient manner without having to resort to fixed amounts of resources. These characteristics must also be managed in a dynamic way, continuously adapting to the needs of the system, without wasting the resources available, and using the most appropriate ones where appropriate.

Therefore, a FOG layer is proposed that can offer all these requirements for the correct operation of the IoT infrastructure, through the automation and dynamic control of the scaling of resources, taking into account the characteristics of the elements belonging to the system, and accommodating the distribution of messages to be processed based on the capabilities of each one.

3. Background and problem definition

The sensorisation implemented at the University of Alicante is a constant challenge, in which a robust and efficient infrastructure is required to collect, process and analyse data effectively. We are working on the development of a platform for the management of data from different types of sensorisation, from measurements of water or electricity consumption, climatology data, to measurements inside the common rooms, such as CO2, temperature or humidity. The architecture that supports this infrastructure is based on the separation of processes, in a manual and controlled way.

In [36] we described the elementary architecture implemented at the

University of Alicante, which consisted of a series of static processing nodes, each dedicated to managing specific types of sensor data. This static allocation approach, although simple to implement and maintain, led to significant inefficiency problems due to the inherent variability of IoT workloads. The number of messages requiring processing varies substantially according to temporal patterns (night versus day, weekdays versus weekends, academic periods versus holidays), as well as the event-driven nature of many sensors, which only transmit data when specific conditions are detected, such as door access or emergency situations. This imbalance in the number of messages to be processed also causes processing nodes to consume computational resources irregularly, as shown in Fig. 1.

In addition to the computational wastage that this situation produces, it is also reflected in the energy consumption involved. Currently our infrastructure is made up of three processing nodes (described in detail in section 7), and if we measure the Watts (W) consumed for each hour of message input, each node has a very different usage cost. As can be seen in Fig. 2, currently all the available nodes are used throughout the day, without any specific usage criteria, to the point of having all the nodes processing few messages, when it would not be necessary for several of them to be switched on.

En conclusión, se desperdiciaron una gran cantidad de recursos para garantizar que la capacidad de la infraestructura fuera suficiente. Además, se observó que los nodos realizaban procesos informáticos desequilibrados, de modo que algunos nodos (los de menor potencia) podían estar al 100 % de su capacidad, mientras que otros apenas se utilizaban.

Este sistema destacó ciertos requisitos que deben cumplir las soluciones posteriores: en primer lugar, la solución debe garantizar flexibilidad para adaptarse a diferentes entornos y requisitos sin pérdida de eficiencia; en segundo lugar, una mayor eficiencia en el uso de hardware, software y recursos humanos, lo que permite una mejor eficiencia energética; tercero, debe permitir la escalabilidad, de modo que pueda soportar escenarios complejos con diferentes cargas de trabajo, manteniendo al mismo tiempo las propiedades del sistema.

These requirements guided the design of a second FOG architecture, this time dynamic, together with an elementary SFMP protocol [14]. The architecture consisted of two distinct levels: a control level, responsible for monitoring the state of the system and making allocation decisions; and a processing level where the acquisition and processing of actual data takes place. The control level contains two new components: the coordinator node, which acts as the central intelligence of the system, continuously monitoring the status of both the balancer and all registered processing nodes to make optimal decisions about workload distribution; and the balancer node, which acts as a traffic routing engine, receiving all incoming messages from IoT devices and gateways and forwarding them to the appropriate processing nodes according to the distribution plan calculated by the coordinator. At the processing level, the processor nodes retain their original data acquisition and processing logic, but now operate under dynamic control, responding to activation and deactivation commands and reporting their status through a standardised management interface. Communication between these components for control and management purposes occurs through the elemental SFMP, which we developed specifically for this architecture. This protocol prioritised simplicity, reliability and minimal overhead.

However, this version of the architecture had several problems. Firstly, the distribution of processing load equally among the nodes is not always the most desirable; sometimes it is necessary to be able to indicate the order of use of resources. Secondly, it lacked robust error control to ensure system availability, since when any of the control nodes fails (coordinator or balancer), the entire system becomes paralysed. Finally, it was also detected that the necessary security mechanisms to protect the distributed system had not been considered during the design.

4. Design of the solution

To resolve the problems mentioned above, substantial improvements to the original, basic architecture [14] are proposed, with several important additions. First, it seeks to automate and customise the priority of resource use. Second, it must maintain high availability through error control mechanisms. And third, it must improve security through the uniform implementation of protection mechanisms for all elements of the system.

To meet these requirements, the following steps will be taken: modifying the resource allocation algorithm of the coordinating node, introducing a new variable called preference that will resolve the need for priority in the choice of resource use; expanding the SFMP protocol along with the elements of the architecture to support fault tolerance; and introducing security mechanisms on the protocol and elements to protect their information and integrity.

4.1. Load distribution algorithm: NPC and preference

The algorithm originally used by the Coordinator to distribute the workload employs the Node Processing Capacity (NPC) indicator [14], which was communicated to the coordinator node during the REGISTER operation, along with its IP and name. In the new system, this REGISTER operation is extended, and each node is defined by the tuple pni = {NPC, IP, name, Preference}. The new Preference parameter is a crucial feature introduced to optimise resource selection priority. It is a value between 0 and 1, where 1 indicates maximum preference for use and 0 indicates low preference. To determine which nodes should be active according to a given sorting criterion, the Coordinator first sorts the list of available nodes by Preference (prioritising values close to 1) and then by NPC

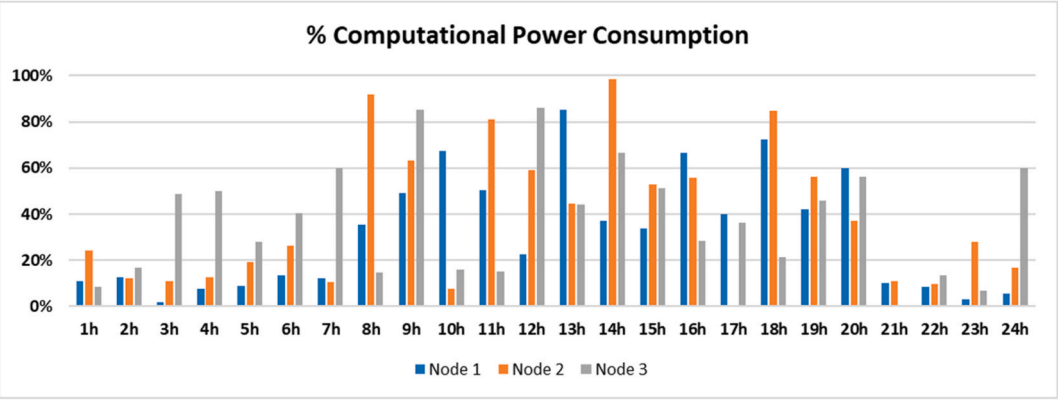


Fig. 1. Computational power consumption at each node when processing messages.

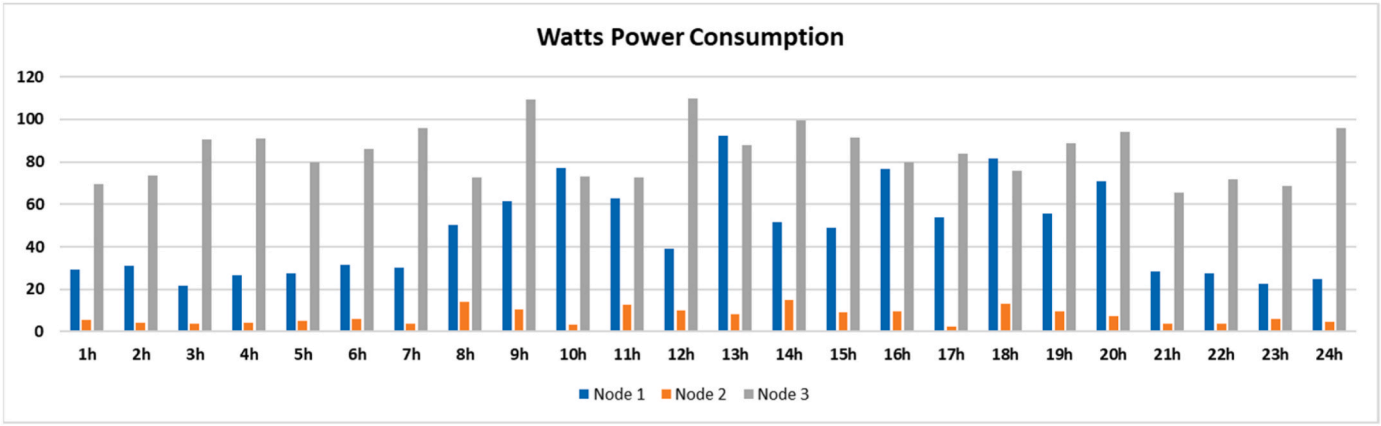


Fig. 2. Watts power consumption per hour of use.

(from highest to lowest). This parameter allows the administrator to prioritise the use of specific nodes (e.g. those with better energy efficiency or lower operating costs), even if they are not the most powerful. Preference is essential so that, in the face of variable demand, the system sequentially activates the most desirable resources (such as Node 1 and then Node 5 in Scenario B, which will be explained later), minimising computational waste and achieving significant energy savings (observed between 40 % and 65 %).

5. Simple FOG Management Protocol - SFMP

This protocol has been developed exclusively to solve the communication between the components of this new architecture. The communication between the different elements of the architecture is carried out by means of a connection-oriented communication protocol, specifically designed for our proposal, in which each request message has an associated response message. Request messages are messages in which one component requests another component to perform an operation. Response messages are messages confirming that the previous message has been received by the indicated receiver. The internal structure of each message shall be as shown in Table 1.

The operations supported by Operation are as shown in Table 2. These messages will be exchanged between the coordinator and the balancer, or between the processors and the coordinator. In Fig. 5 this is represented by the dashed lines.

- GETSTATUS: The coordinator node notifies the balancer node to send its status.
- INFOSTATUS: Information to be sent by the balancer or coordinator node to the primary or alternate coordinator, with the required data. The type of data to be sent will depend on who generates the INFOSTATUS message.
  - o Balancer: The packet shall contain the total number of messages per second arriving to the message queue, plus the name of each processor node to which it sends data, the load distribution assigned to that processor, and the address where the processor is located.
  - o Coordinator: This packet will be sent to another coordinator, with the list of active processors in the infrastructure, that is, the resources it has. Among the information sent is its name, capacity, URL of the IP address where it is located, the preference to be used, the number of AYA messages remaining (AYACOUNT), and finally the UID of the AYA message that is arriving (AYAUID). The functionality of this type of message will be defined later.
- SETCONFIG. The coordinator node sends the new configuration to the balancer node, indicating the load to be assigned to each active node.
- REGISTER: The processor node notifies the coordinator node that it is available for use in the infrastructure. This element may or may not be used when the load is redistributed, but it is added to the coordinator node's list of available resources.
- UNREGISTER: The processor node notifies the coordinator node that it is going to be deactivated from the infrastructure, so that the coordinator node must redistribute the load of the infrastructure without taking this node into account. Once the new load redistribution among the other nodes is completed, the processor node in question is removed from the list of available resources.
- ARE YOU ALIVE: This message is used to let an element know if another element is up and running, i.e. alive. Its use will be explained in detail in section 4.3.
- RESULT: Any of the infrastructure elements shall send a reply message without content, as an acknowledgement of receipt of the first message.

Table 1
Content of SFMP messages.

Operation	Data
UID	Message identifier
Type	Type of message, request or reply
Operation	Action to be taken by the receiving element of the message. For messages that are response messages, this parameter shall contain a RESULT, while for request messages, there are other six types of operations
Data	Objects of information to be sent

Table 2
Table of operations.

Operation	Data	Type
GETSTATUS	{urlCoordinator: https://IP:PORT}	REQUEST
INFOSTATUS	**Balancer to Coordinator schema** {receivedMessages: value (number), balancerList: [{name: name n (string), load: value(number)}]} **Coordinator to Coordinator schema** {name: name n (string), capacity: value(number), url: http://IP:PORT, preference: value (number), AYACOUNT: value (number), AYAUID: value (number)}	REQUEST
SETCONFIG	{name: name n (string), load: value(number), url: http://IP: PORT}	REQUEST
REGISTER	{name: name n (string), capacity: value(number), url: http://IP: PORT, preference: value (number)}	REQUEST
UNREGISTER	{node: name n(string)}	REQUEST
ARE YOU ALIVE?	NULL	REQUEST
RESULT	NULL	REPLY

5.1. Coordinator - balancer communication

The following figure, Fig. 3, shows the exchange of messages of the communication protocol between the coordinator node and the balancer. This message exchange will occur on a scheduled basis, with a time period defined in "UPDATE_CONFIG", which we initialise at 300 s, or for each change in the system that is registered. A change in the system occurs when a new processing node becomes part of the infrastructure, or when a processing node is no longer accessible (discussed in the following subsection, 6.2, on communication between coordinator and processor).

As shown in Fig. 3, when the coordinator is about to start communication, either because a change has occurred, or because the time period stipulated in "UPDATE_CONFIG" has elapsed, it requests the balancer to know its status (message 1). The balancer responds with a REPLY to this request (message 2). In case a REPLY to any REQUEST does not occur, the sender of the REQUEST repeats the sending of the request. The time for resending the message is defined in "WAIT_AYA", initialised to 10 s. This sending would be repeated up to a maximum of 3 attempts. After confirming the reception of the message, the balancer sends its status to the controller (message 3) and receives a confirmation REPLY (message 4). The coordinator, with the information received on the status of the balancer, plus the internal information on the available processor nodes, calculates the appropriate configuration and transmits it to the balancer (message 5), and receives the corresponding REPLY (message 6). Messages 5 and 6 would only be performed if there is a status update.

5.2. Coordinator - processor communication

This communication is carried out using the REGISTER and UNREGISTER operations [14]. A new parameter, Preference, has been introduced into the REGISTER message, which is used in the coordinator algorithm discussed in section 4.1. Fig. 4 shows the message sequence for both actions and how their contents are structured with the new parameter. The essential functioning of the messages has not changed, but their content has.

It is important to note that REGISTER and UNREGISTER messages (Fig. 4) are proactive messages sent by the processing nodes to inform the coordinator about their availability. These messages trigger actions at the coordinator, such as redistribution of the workload. RESULT messages are generally used as acknowledgement messages to ensure the reliability of the communication.

5.3. SFMP protocol error handling

The proposed solution uses the redundancy of elements in the infrastructure, adding two additional elements to those explained above, a substitute coordinator and a substitute balancer. The task of these two elements will be to resume the activities that the main process was carrying out in case it is not active. These substitute elements will be located in other nodes of the infrastructure, different from the nodes where the main balancer and coordinator are located, and which have sufficient resources to temporarily assume the functionalities of either of them.

For the correct operation of the protocol, a series of actions have been established as safeguards for possible errors that may arise in the infrastructure, such as the crash of one of the processes described. These safeguards include defining which of the infrastructure elements realises that this process is down, and what should be done accordingly. Five scenarios can be distinguished: no substitute element is active; substitute coordinator is active; substitute balancer is active; both substitutes are active; and finally, loss of a processor. Table 3 shows the list of messages used explicitly for error control between substitute and main components. For simplicity, only the messages in Table 3 that are directly involved in the scenario will be shown in Figs. 5 and 8–11.

5.3.1. Scenario 1. No substitute element is active

First, we have the original case, where none of the substitute elements are active. Within these scenarios, two types of messages appear in addition to those shown in the previous figures: the "ARE YOU ALIVE?" (AYA) messages; the "SETCONFIG" message which is now duplicated; and the "INFOSTATUS" message for the configuration of the coordinators.

Fig. 5 shows the origin and destination of the "AYA" messages, the green arrows, with message numbering [1–3]. This numbering does not imply sequentiality in the messages, since they can occur either all at the same time or discontinuously. Message 1 from the coordinator to the processor will check the status of the processor, so it checks if it is still active. The processor will send an affirmative REPLY if it is still running. This message exchange is maintained for messages 2 and 3 also for their corresponding senders and receivers.

In the case of the purple arrow, message 4, this is a replica of the "SETCONFIG" message that was only sent to the main balancer, and now it is also sent to the substitute balancer. The coordinator node sends this distribution both to the main balancer, which needs this information to send the messages accordingly, and to the substitute balancer, since in the event that the main balancer is no longer operational, the balancer node can maintain the load distribution that was being used, until the next change in the message distribution is made.

It has been defined that the sending of "AYA" messages is carried out according to the value in "NEXT_AYA", initialised to 60 s, so that when an "AYA" message gets a response from the receiver, it means that it is still in operation and the infrastructure will be maintained with the corresponding active elements. In case no response is received within the time defined in "WAIT_AYA", initialised to 10 s, a total of 3 messages

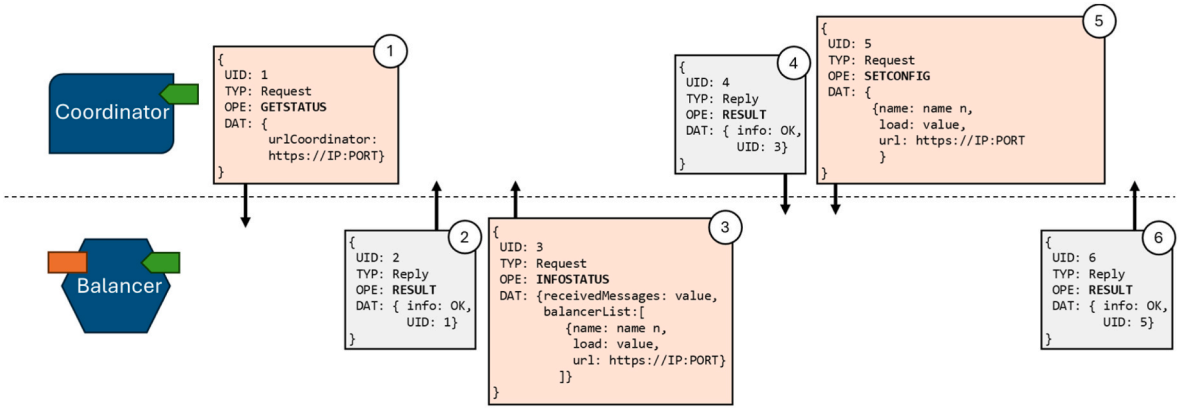


Fig. 3. Exchange of messages and arguments in the Coordinator - Balancer communication.

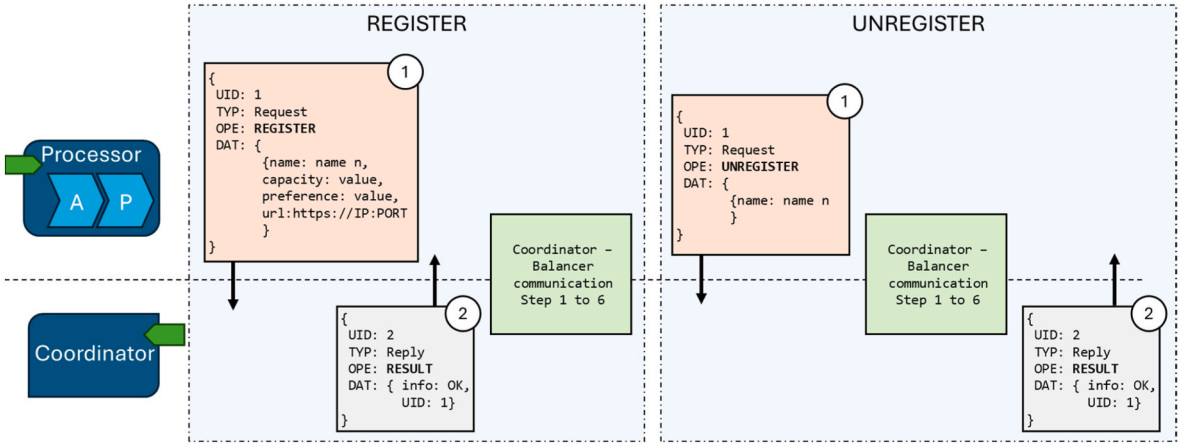


Fig. 4. Message exchange and message contents for "REGISTER" and "UNREGISTER" operations.

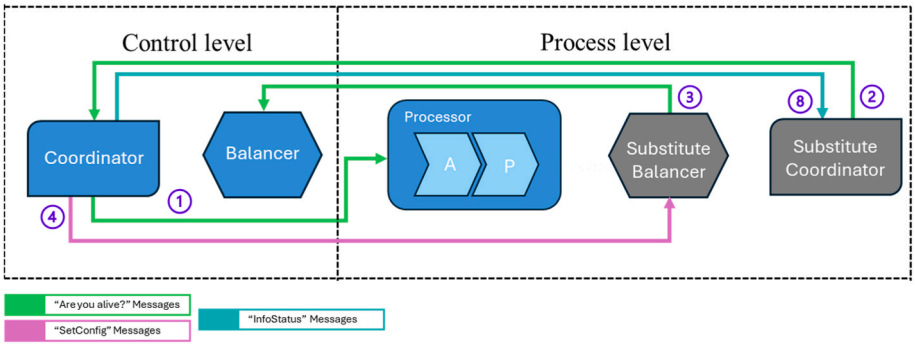


Fig. 5. Scenario 1. No substitute element is active.

Table 3
Table detailing the error control messages exchanged between the components of the FOG architecture.

ID Message	Sender	Receiver	SFMP Type of Operation
1	Main Coordinator	Processor	ARE YOU ALIVE?
2	Substitute Coordinator	Main Coordinator	ARE YOU ALIVE?
3	Substitute Balancer	Main Balancer	ARE YOU ALIVE?
4	Main Coordinator	Substitute Balancer	SETCONFIG
5	Substitute Coordinator	Processor	ARE YOU ALIVE?
6	Substitute Coordinator	Substitute Balancer	SETCONFIG
7	Substitute Coordinator	Main Balancer	SETCONFIG
8	Substitute Coordinator	Main Coordinator	INFOSTATUS

are sent every "WAIT_AYA" seconds. If no response is received after these sending's, it is assumed that the receiver of the "AYA" message (coordinator, balancer or processor) is not active, and it is necessary to react to this incident.

The structure followed by these messages is shown in Fig. 6, where any element can receive an "AYA" message, while the sending of these messages is done by all elements of the infrastructure except the processors. Fig. 6 also shows when there is a response to the "AYA" or when a forwarding is necessary.

As shown in Fig. 7, the exchange of messages that occurs from the "AYA" message sent between the coordinators, changes with respect to the other types of messages. In the case of message 8, of type "InfoStatus", shown above in Fig. 7 with the turquoise arrow, when the main coordinator receives an "AYA", in addition to the corresponding REPLY, it will also send an "INFOSTATUS" to the alternate coordinator. The alternate coordinator thus obtains the list of active processors in the infrastructure, along with their capacity, preference and IP address in other data.

5.3.2. *Second scenario. Failure of the main coordinator and activation of the substitute*

For the second scenario, Fig. 8, it is shown that the main coordinator does not respond. After the three communication attempts by the substitute coordinator to the main coordinator have been exhausted, it is considered inactive in the infrastructure, and the designated substitute coordinator continues to work.

As can be seen in Fig. 8, some messages that were in the previous scenario are maintained, namely messages 2 and 3, indicated by the dotted arrows. The substitute balancer continues to send the corresponding activity check messages to the principal, and in the case of the substitute coordinator, it will continue to send messages every "NEXT_AYA" seconds to the principal, until it is operational again to resume its functions in the infrastructure.

In the case of message 8, the "INFOSTATUS" communication is not dotted because, although it is the same type of message, the address from which this message is sent changes, due to the fact that now it is the substitute coordinator who must send the list of active nodes in the system to the inactive main coordinator, for when he resumes his functions.

In addition to the communications that remain the same, some new ones appear. Starting with the "AYA" messages, since the main coordinator is inactive, it will now be the substitute that sends the activity check messages to the processor nodes (message 5). As for the

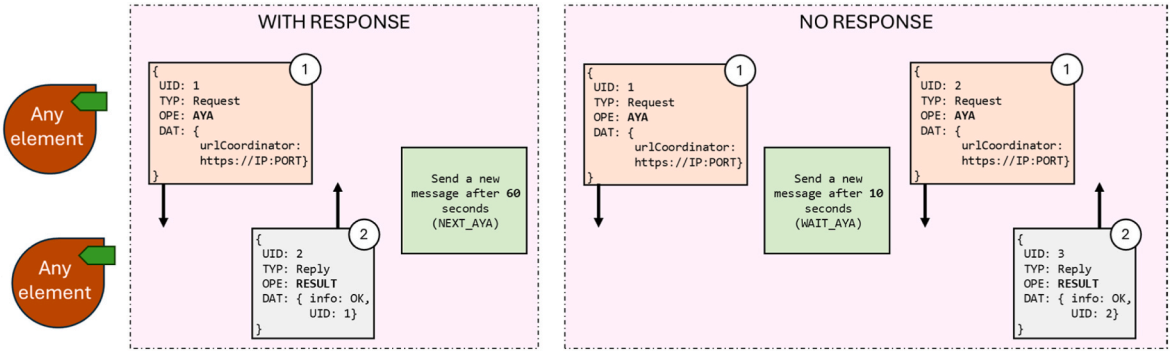


Fig. 6. Exchange of "AYA" messages, with and without response.

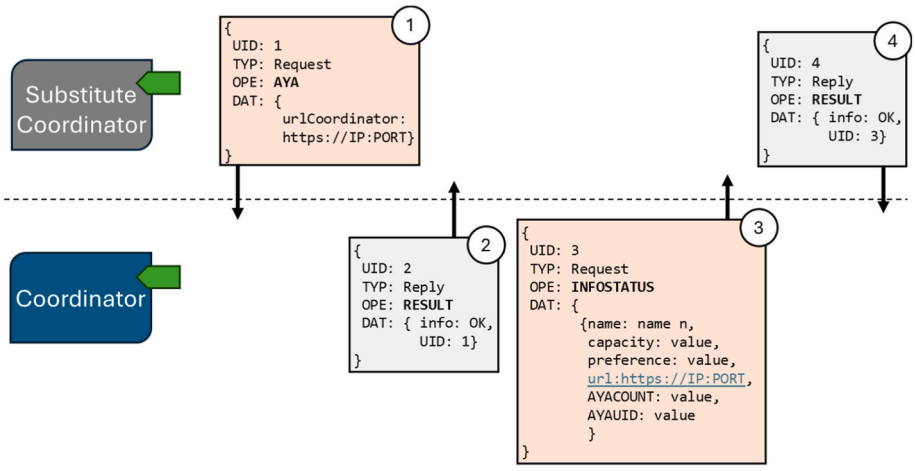


Fig. 7. Exchange of "AYA" messages between coordinators.

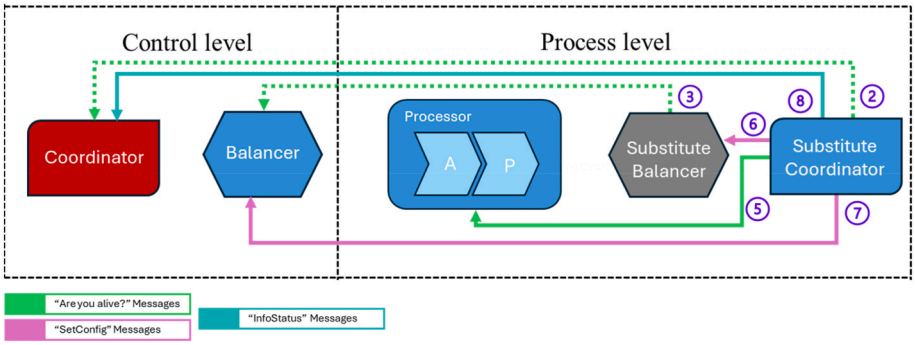


Fig. 8. Second scenario. Failure in the main coordinator and activation of the substitute.

"SETCONFIG" messages, messages 6 and 7, the message load distribution currently in use will be sent to both balancers, to the main one because it needs it to forward messages accordingly, and to the substitute in order to maintain the message forwarding flow as it was at the last moment before the main balancer went down.

5.3.3. Third scenario. Failure of the main balancer and activation of the substitute

In the third scenario, in which the substitute balancer is activated, the messages of the previous cases are maintained, messages 1 to 4, together with message 8, as shown in Fig. 9. No new messages appear, since the only thing the balancer does is to transmit the incoming data from the device layer and respond to the requests sent by the coordinator. These communications are those explained in previous sections using the SFMP protocol.

5.3.4. Fourth scenario. Failure of the main coordinator and balancer. Activation of the substitutes

In the fourth scenario, none of the main elements are active, so the substitutes start working. As can be seen in Fig. 10, no new communications appear that have not been explained in the previous scenarios, so all connections appear in the dotted format described above. In this scenario, both substitutes continue to monitor the status of the principals, waiting for them to become active again in order to resume their functions, while they carry them out.

5.3.5. Fifth scenario. Failure of a processor. Elimination of the resource

In the fifth scenario, when a processor ceases to be operational in the

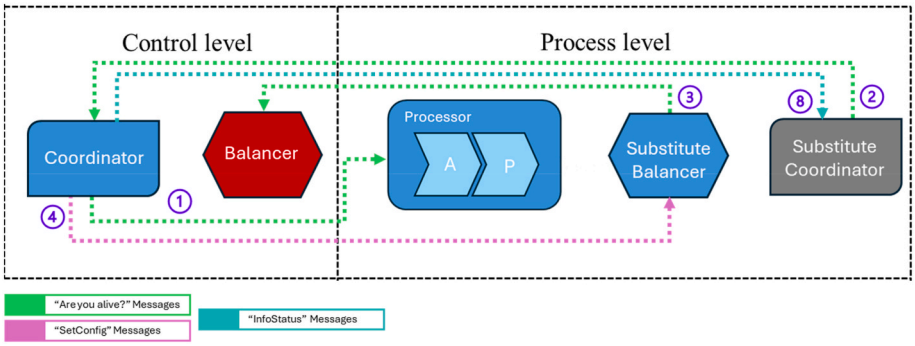


Fig. 9. Third scenario. Failure of the main balancer and activation of the substitute.

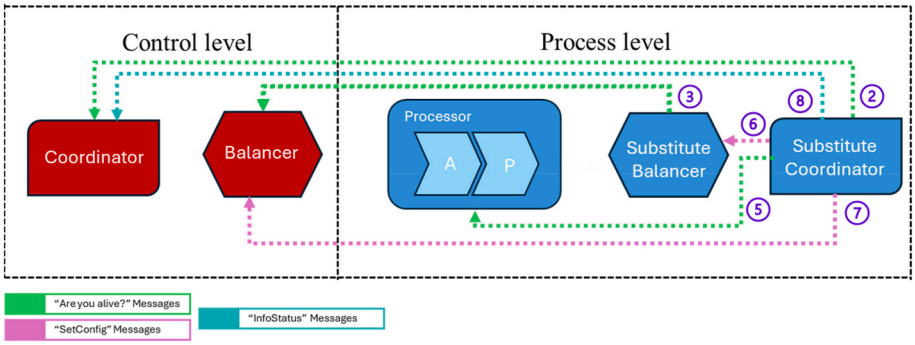


Fig. 10. Fourth scenario. Failure of the main coordinator and balancer. Activation of the substitutes.

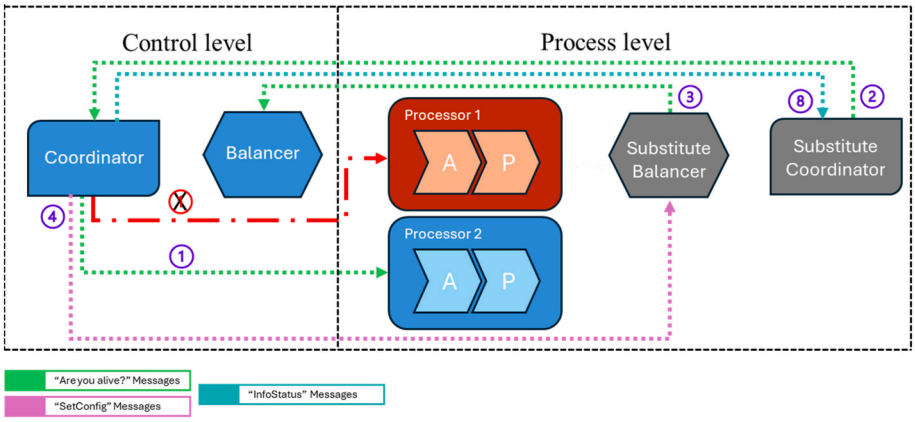


Fig. 11. Fifth scenario. Processor failure. Removal of the resource.

infrastructure, it will be noticed by the coordinator active at that time, who is responsible for sending it "AYA" messages periodically. When no response is received, it considers that it is not available to continue processing data and decides to remove it from the list of available resources. In this way, the active coordinator will initiate the "UNREGISTER" protocol, in which it will perform a new load distribution among the rest of the available nodes in the infrastructure and finally send the new configuration to the balancers. Since it is removed from the list of resources, the communication arrow "AYA", assigned as message 1, no longer occurs, because it will no longer be an active resource in the infrastructure, nor will it wait until it is functional again to use it, as shown in Fig. 11. When it is added back to the system through a "REGISTER", it will be taken into account again and the load will be distributed accordingly.

5.4. *Restoration of an element*

Having explained all the scenarios, let's describe how they return to normal functions. After going through a situation in which the coordinator or balancer have been inactive, they do not initially have the configuration that applies to the infrastructure at that moment, i.e., neither the active processors that are present, nor the load distribution that the balancer is using. In order to obtain this information and resume its functions, the following message exchange is performed, as shown in Fig. 12.

First, any of the substitute elements that were active in the infrastructure maintain "AYA" messages to the inactive main elements. Once they receive an affirmative REPLY that they are back up and running, i.e. fully restored, each of the substitute elements sends to its corresponding main counterpart an "INFOSTATUS" message with the currently used configuration, marked with the blue, undotted arrow in the figure.

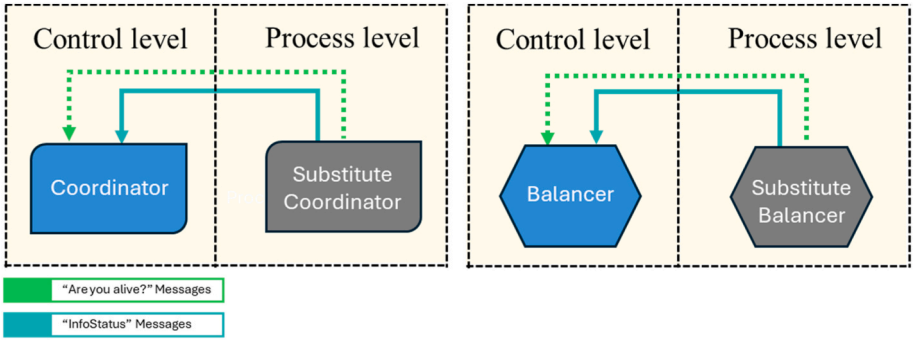


Fig. 12. Restoration of elements. Sending the current configuration via "INFOSTATUS" message.

As explained above, the content of the "INFOSTATUS" will vary depending on who sends it, as shown in Fig. 13. For the balancer, it will send the total incoming messages from the devices, plus the message load used on each processor. For the coordinator, it shall send the list of processors along with their internal characteristics for each processor.

5.5. Security issues

When creating new distributed architecture that includes new elements, it is necessary to analyse its vulnerabilities, and the necessary mechanisms have been designed to secure the system.

5.5.1. Communications

As this is a distributed system with multiple nodes and a new communication protocol, it is necessary to protect communications between each element of the system. To this end, communications are encrypted using TLS 1.3 channel security through certificates. This will allow us to hide the information in the messages from third parties thanks to the encryption that is performed. It also authenticates the elements (balancer, coordinator or processor) involved in the communication, guaranteeing the integrity of the messages. These certificates are emitted by external official entities.

5.5.2. Identity management

The system has a measure for verifying the identifiers of the processor nodes that can form part of the assignable resources. When a processor node registers in the system, it provides a Name and an IP:Port. The controller has a list of the elements that can be registered, so it verifies that both fields belong to a permitted element. This list is provided by the infrastructure administrators and contains only valid processing nodes, so that if any external node attempts to register in the system with an unknown IP:PORT and/or Name, it will be rejected. In addition, the Name will be validated through the node certificate to ensure identity.

5.5.3. Centralised security

As designed, the system architecture implements a centralised security model where the coordinator acts as the control and management point for all system security. This approach allows for granular and consistent control of security policies across the distributed network.

6. Characterisation and analysis of the proposal

The SFMP (Simple FOG Management Protocol) is designed to manage communication between the components of a dynamic, auto-scalable FOG architecture, with the aim of optimising the use of resources and ensuring system availability. In the following, the impact of the SFMP protocol on system communications will be analysed, including considerations on delays and efficiency achieved.

6.1. Impact of the SFMP protocol on system communications

The new system implements a series of messages, both for system efficiency control and for error control in case of incidents with the balancer, controller or processors. The number of additional messages to be sent can be calculated as follows:

- Messages in infrastructure control:
  - o REGISTER messages, M_R: 1 message per processing element added to the system.
  - o UNREGISTER messages, M_U: 1 message for each item that is no longer available in the system.

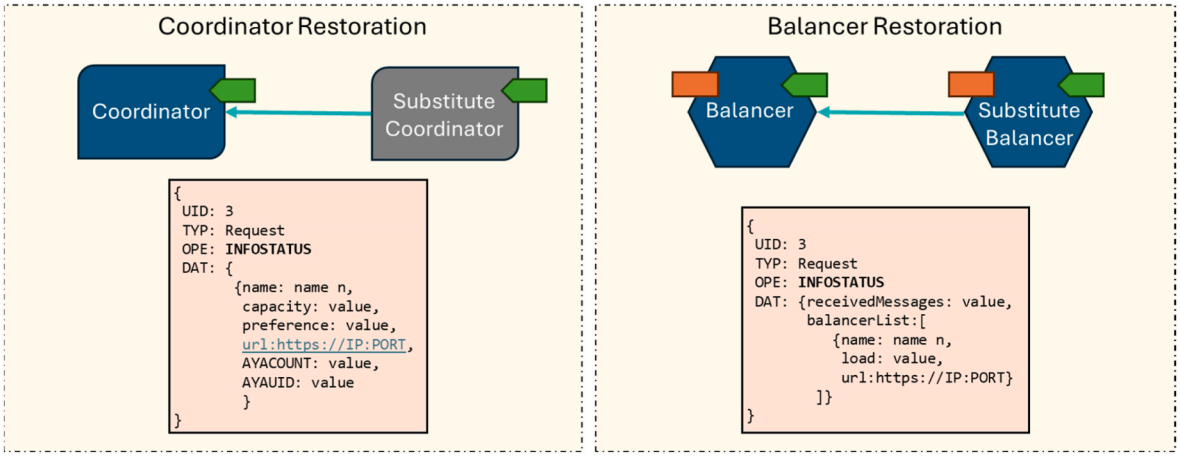


Fig. 13. "INFOSTATUS" messages according to the requesting sender.

- GETSTATUS messages, M_G: 1 message every UPDATE_CONFIG seconds, 1 message after every REGISTER, and 1 message after every UNREGISTER.
- INFOSTATUS messages, M_I: after each GETSTATUS.
- SETCONFIG messages, M_S: after each INFOSTATUS message. Note that the INFOSTATUS message is sent by both the main element ($M_{S\ PRINCIPAL}$) and the substitute ($M_{S\ SUBSTITUTE}$).
- RESULT messages, M_T: after each of the above messages.

- Communication error control messages in the infrastructure:
  - AYA messages, M_A: 1 AYA message is sent every NEXT_AYA seconds, from the active coordinator to each processor, 1 AYA message from the substitute coordinator to the main coordinator, and 1 AYA message from the substitute balancer to the main balancer. When the substitute coordinator is activated, the AYA message to the processors is sent by this new coordinator. When a response fails, the AYA is resent, and this can happen 2 more times every WAIT_AYA seconds.
  - INFOSTATUS messages, M_I: after each AYA message from the substitute coordinator, the primary coordinator sends an INFOSTATUS message to the substitute coordinator. In addition, after a recovery, the substitute coordinator sends an INFOSTATUS to the main coordinator to restore the system configuration.
  - SETCONFIG messages, M_G: after sending a SETCONFIG to the primary balancer, send another SETCONFIG to the alternate.
  - RESULT messages, M_T: after each of the above messages.

Taking into account the above definition, and in the case of faultless operation, where the number of processors in the system is NP, in an estimated time of Time seconds, the number of messages sent can be calculated as:

$$\left(\left(\frac{Time}{UPDATE_CONVIFG}\right)*A+\left(\frac{Time}{NEXT_AYA}\right)*B+N_P*C\right)*2 \quad (3)$$

- $A = M_G + M_I + M_{S\ PRINCIPAL} + M_{S\ SUBSTITUTE} = 4$ messages
- $B = AYA*3N_P + AYA(coordinator) + AYA(balancer) + INFOSTATUS = 6$ messages
- $C = M + M_{GI} + M(S_PRINCIPAL) + M_{S\ SUBSTITUTE} + M_R = 5$ messages

The formula multiplies by a value of 2 at the end because each message has a response RESULT.

If we take a practical example of this definition, in a system with 3 processors, during 1 h approximately (12*4 + 60*6 + 3*5) * 2 = 846 messages would be sent. This number of messages in a system where hundreds of thousands or millions of messages per hour are being processed, such as in IoT environments, is a completely acceptable value as it is practically negligible.

6.2. Error handling efficiency analysis

At this point, we will carry out an analysis of the efficiency obtained after the implementation of the SFMP protocol. For this purpose, the following terms are defined:

- Response time ($T_{Response}$): time it takes for a substitute node to react to a fault, *NEXT_AYA + N*WAIT_AYA*.
- Activation time of the substitute ($T_{Activation}$): time taken for the substitute element to start up, which is considered negligible, so it will be defined as constant *C*, because it is switched on and ready to start its tasks at any time.
- Repair time (T_{Repair}): This is an external constraint, as it will depend on the actions to be taken (some examples may require physical actions such as changing the board, resetting the system networks, internal actions by administrators, etc.). This is a time that cannot be known exactly, so it will be catalogued with a value *X*, assuming that it can be either a short time or longer than expected.
- Restoration time ($T_{Restoration}$): the time it takes for the secondary node to relinquish control to the primary node once the latter is active and fully operational again, in general, the time defined in *NEXT_AYA*.
- Recovery time ($T_{Recovery}$): it is the final value among all the elements discussed above, formally defined as:

$$T_{Recovery} = T_{Response} + T_{Activation} + T_{Repair} + T_{Restoration} \quad (4)$$

$$T_{Recovery} = (NEXT_AYA + (N*WAIT_AYA)) + C + X + NEXT_AYA \quad (5)$$

The activation of any of the substitute elements does not have a major impact on message processing, i.e. the response times for any of the standby nodes to become operational are the seconds set for NEXT_AYA and WAIT_AYA. These times are configurable and can therefore be reduced as much as necessary as long as the infrastructure supports it. The performance of the system is maintained efficiently as the secondary nodes remain alert and waiting for a possible failure of any of the main elements, so that the data processing flow continues after a lapse of a few seconds.

6.3. Comparison of the proposed solution with traditional solutions

This section introduces a comparative table, Table 4, which summarises the characteristics of the main traditional solutions analysed in the "Related Work" and highlights the differences in performance compared to our proposal (SFMP column). As can be seen, our proposal tries to use only the necessary resources according to the load to be processed, without predictions that may reserve resources that may later become unused, and in an adaptive way, since the coordinator analyses the incoming traffic together with the computational capacity of our nodes to choose the best possible configuration at any given moment.

Section 7.4 analyses how each of these techniques works using several real-life scenarios, such as a university campus with multiple locations and nodes, and also analyses the non-viability of traditional techniques and the need for our new strategy.

6.4. Limitations

The new architecture introduces a balancing element responsible for directing data flow to the most appropriate node, which implies an increase in latency compared to the original system. Previously, each processor node connected directly to the message broker, obtained the new message, and processed it. The latency of this action was solely the transfer time between the broker and the processor. In the new architecture, this transfer time is consumed by the balancer, which is the node that obtains the message from the broker. Additionally, once the balancer has decided where to redirect the data, it sends it to the corresponding processor node. This increases the total latency time due to the addition of this communication time between the balancer and the processor. This communication is carried out via Rest API requests, as will be seen later in the following implementation scenarios, which introduces only a few milliseconds. This additional latency will be affected by different external factors such as the physical characteristics of the machine, whether the communication passes through external public networks to reach distant nodes, and the available bandwidth. However, we must consider the context of our application, sensorisation through IoT. Most devices have long latencies, such as those that periodically transmit the environmental conditions of a classroom every 15 min, while others with short latencies, such as door detection or fire detection, have latencies of around 1 s. It is therefore considered that the increase of milliseconds on these latencies is negligible. It should be noted that this consideration may not be valid in other scenarios where the time required to respond to a message is important, but this is not the case for us.

Another important aspect is the recovery time in the event of system

Table 4
Summary table of the contributions of the SFMP protocol compared to other existing solutions.

Schema	PPPA	ODPPA	ODPSBA	SFMP
Operation	Reserve resources based on IoT traffic predictions for immediate processing.	Allocates resources when receiving IoT requests, prioritising the speed of allocation to the nearest FOG node	Optimises resource allocation for a set of IoT requests, prioritising long-term efficiency	Automate and dynamically control the scaling of the FOG layer to suit the needs of the IoT system, ensuring minimum resource usage
Time at which resources are provisioned	Pre-arrival of IoT applications, based on predictions.	Upon arrival of the IoT application.	After a batch of IoT applications accumulates	Dynamic: resources are adjusted according to demand in real time, through a coordinating node.
Allocation of resources	Immediate access to reserved resources, assuming availability.	Immediate to the nearest FOG node, without considering the burden or cost of other nodes.	Optimal for the batch of requests, looking for the best metrics (latency, cost), before starting processing.	Dynamic: a balancer node distributes the tasks among the processing nodes as directed by the coordinator. The distribution is based on the processing capacity of each node.
IoT request processing	Requests are processed immediately on the reserved resources.	If the nearest node has no resources, it can be divided into tasks, propagated to other nodes or to the cloud. FIFO or delay priority can be used.	Applications are accumulated in small batches for joint allocation. Allocations persist throughout the life of the application.	The balancer receives messages from the sensors and forwards them to the corresponding processing node. The processing nodes are reactive in nature.
Latency weighting	It seeks to minimise latency through early reservation of resources and immediate processing.	It prioritises initial latency by mapping to the nearest node, although it can lead to high latency if the node becomes saturated.	Batch allocation may introduce initial latency due to the backlog of requests and optimisation but offers efficiency in the long run.	Minimises latency through efficient load distribution and use of each node's processing power
Weighting of costs	Costs can be adjusted according to the likelihood of long-term use of an IoT device. High utilisation of the FOG is sought.	Generally, it does not consider the costs of resource utilisation, prioritising initial latency and speed. This can result in sub-optimal costs in variable cost environments.	It actively seeks to optimise costs while meeting the requirements of IoT applications.	It optimises the use of resources to minimise costs, activating processing nodes only when necessary. Prioritise the use of the most powerful nodes.
Types of IoT environments in which it is applied	Environments that require immediate processing and have widespread and predictable use of FOG.	Dynamic environments needing immediate support and unpredictable usage.	Static environments that need long-term, allocation-efficient support, even though they can tolerate higher initial latency	Adaptable: designed for IoT environments with varying demands, both for static and dynamic applications, always looking for efficiency.
Examples of Models/ Techniques Used	Analysis of historical resource use and QoE, allocation of resources according to probability of use.	FIFO and delay priority processing policies.	Mixed integer linear programming (MILP), algorithms for allocation optimisation.	SFMP protocol for communication and management of the FOG layer, load sharing algorithm based on the processing capacity of each node (NPC).
Necessary Assumptions	Reserved resources are sufficient, any FOG node can satisfy the required latency.	The nearest node is sufficient for the required latency, it can be propagated to other nodes or to the cloud.	Static set of IoT requests, guaranteed bandwidth availability, awareness of all resources.	The infrastructure is able to detect failures and reconfigure itself automatically. It is based on continuous monitoring of the nodes. The system seeks efficiency in the use of resources without wasting them.

errors. As seen in section 5.3, there are multiple failure scenarios, which will define the exchange of messages between elements. AYA messages are key to detecting a system failure and subsequently resending until confirmation that a node is down. The AYA messages emitted by the controller have been defined with a cadence of 60 s plus the resending of 3 messages every 10 s. This makes a total failure detection time of 90 s. However, if earlier detection is necessary, it is possible to configure these values to shorter times, although this will entail a greater load of SFMP protocol messages. As seen in section 6.1, the number of messages generated by the system under this configuration barely reaches 1000 msg/h, which is an amount of data that does not affect the system that is managing millions of messages per hour. Decreasing the node failure detection time with the consequent increase in SFMP protocol messages would also not affect the system.

6.5. Cost estimation

Innovation always comes with an associated cost, and in this case, where such a profound change is being made to the functioning of a system, it is necessary to analyse what that cost might be.

Firstly, the development of a new system generates implementation costs. The proposed architecture offers significant advantages by completely reusing the existing hardware infrastructure. No additional investment in computational resources is required, as the system redistributes and optimises the use of available equipment, transforming the implementation model without increasing the technology pool. With regard to the new software, it reuses the heaviest business logic modules from the previous system, such as data processing, data capture by the broker and even the connection of the system to the broker.

Being able to reuse existing code streamlines development and reduces the need for investment. Once up and running, its operating and maintenance costs must be considered. Our proposal generates substantial savings in technical human resources. Centralised management and monitoring drastically reduces operational complexity compared to the traditional model of multiple distributed applications operating independently.

Technicians experience a significant reduction in the hours spent on maintenance, supervision and incident resolution tasks. In terms of operational costs, the system provides significant benefits in terms of hardware lifecycle. Intelligent resource management allows machines to remain idle for longer periods, reducing wear and tear and extending their operational life, which translates into lower replacement and preventive maintenance costs.

7. Implementation and validation

The proposed FOG architecture has been validated through its implementation at the University of Alicante, where it is used to manage the sensorisation of various operational areas. The institution's San Vicente campus covers more than one million square metres and includes more than 70 buildings, roads, a desalination plant, lighting and video surveillance systems, extensive landscaped areas and multiple additional services. As previously noted, its characteristics are equivalent to 95 % of small and medium-sized cities with populations of up to 40,000 inhabitants. In addition to the main campus, the University of Alicante has a smaller campus in Alcoy, twelve university centres and

classrooms distributed in different locations, as well as five research centres and museums throughout the Valencian Community.

The Alicante university campus, although it is a single campus, is integrated by multiple IoT infrastructures that use various sensorisation devices, among them: climatological sensors, with data on outdoor temperature, relative humidity, light levels, UV rays, rain; sensors in the interior spaces of the buildings to monitor temperature, humidity, air quality, CO2 concentration, suspended particles, presence of harmful gases; consumption monitoring, where data is acquired on water, electricity and gas consumption; sensors to measure the energy generation of photovoltaic plants; sensors to monitor the use and consumption of the vehicle charging stations; sensors to monitor the use and consumption of the vehicle charging stations; sensors to monitor the use and consumption of the vehicle charging stations; sensors to monitor the use and consumption of the vehicle charging stations; sensors to monitor the use and consumption of the vehicle charging stations; sensors to monitor the use and consumption of vehicle charging stations; WIFI network sensorisation, with information on the users connected to each access point of the campus wireless network; monitoring of the lighting network, with data on lighting systems including outdoor streetlights, signage, courtesy lights in monuments, etc.; and many other various sensors. Fig. 14 shows four of these sensor networks deployed on the campus. Therefore, this validation scenario can be considered as if it were multiple different scenarios from completely different cities.

The data sets used in the scenario described are not limited to the real-time data collected by the sensors implemented, but values are also obtained through datasets uploaded in CSV format, which come from information outside the university's own sensorisation. For example, data from external infrastructures are added to the data extracted from the sensors, such as the water consumption monitored by the drinking water supply companies. Therefore, the information used to validate the correct functioning of the proposed solution is completely varied, coming from infrastructures with different architectures, but which our system can support.

Currently, the volume of data from campus sensors exceeds 2 million records per hour at peak times. This figure is due to the existence of multiple monitoring subsystems, some consisting of thousands of sensors and others of just a few dozen. Data capture intervals vary according to the nature of each measurement: some are taken every 15 min, while others operate every 30 s, depending on the expected volatility of the monitored values. Some sensors are programmed to operate at specific times of the day, such as those designed to record water and electricity consumption or Wi-Fi connections. At the same time, other measurements are activated by specific events, such as access to premises, verification of the status of lighting fixtures, recording of ambient temperature or measurement of humidity in green areas. This operational diversity means that the FOG layer receives a variable and indeterminate flow of messages, which can range from very high to very low volumes.

To validate the proposal, we will demonstrate its implementation in two different scenarios at the University of Alicante. The first will use only the main campus in San Vicente (described in section 3), and the second will extend the main campus to include two other locations, the Alcoy campus and the La Nucia university centre.

7.1. Scenario A

The objective of this test is to verify that the use of resources based on the preference parameter actually generates a prioritised selection of resources according to the system administrator's configuration. This varies greatly from the operating mode shown in Ref. [14]. In that study, the feasibility of dynamic scaling and load balancing was demonstrated, but in the absence of preference, the behaviour of the infrastructures could not take into account the administrator's needs, such as the actual cost of using an infrastructure.

This scenario uses the FOG layer infrastructure implemented on the San Vicente campus, which consists of three physical machines, one with a high processing capacity, the characteristics of which are as follows: Node 1, AMD Ryzen 7 5800X processor, 32 GB RAM, 1 TB SSD and an RTX 4070 graphics card; Node 2 with a Raspberry Pi 4, with an ARM Cortex-A72 and 2 GB SDRAM; and finally, Node 3, medium capacity with an Intel Core i5-12400 processor, 16 GB RAM, 512 GB SSD, and an integrated UHD Graphics 730 graphics card. These three nodes are physically located in three different university buildings on the St. Vincent campus.

At the technological level, the sensorisation has been deployed using LORA technology, so that all the sensorisation will be captured through Mikrotik wAP LR8 access points (APs). In the IoT layer, the opensource platform TheThingStack (TTN) is used as packet concentrator and its



Fig. 14. Interactive map representation of part of the sensorisation at the main campus of San Vicente from University of Alicante.

integration towards the balancer node is done using the MQTT protocol. The entire FOG layer has been developed using Typescript [37] processes executed with Node, and the inter-faces between processes use API Rest. To obtain data, the balancer uses a Publish/Subscribe architecture in MQTT, so that we connect to an existing data queue in TheThingsNetwork platform, using specific credentials (AppID and accessKey) and with them the balancer receives all the sensor packets from the campus.

It should be noted that TTN is an IoT message concentrator platform external to our system, it is completely outside our control, to which only the active balancer node connects via MQTT. When a substitute balancer node starts to perform the tasks of the primary balancer node at the moment it is taken down, no change is made to the TTN configuration. TTN was chosen for the sake of convenience as it is the most widely used platform in IoT environments in Europe, however, another service concentrator such as HiveMQ could be used.

For the processor nodes, Linux has been used on nodes 1 and 3, and the Rasberry PI embedded system on node 2. The processors have been implemented using NodeJS routines managed through PM2.

Virtual machines running on the university's data centre servers are used to establish the coordination and balancing nodes. The physical machines previously mentioned will make up all the other processing nodes. Each of these machines has an NPC and Preference configuration: Node 1 NPC = 2000 and Preference = 1; Node 2 NPC = 500 and Preference = 0.5; Node 3 NPC = 1000 and Preference = 0.2. Infrastructure saturation is prevented by the controller node, which imposes a limit of 80 % of the processing capacity of any node. Specifically, if the queue of messages to be processed exceeds 80 % of the messages handled by the currently active nodes, the controller will activate the next node in the list. Conversely, a node will be deactivated if the number of messages to be processed falls below the 80 % threshold. The results of the tests carried out over 24 h are illustrated in Fig. 15.

The current behaviour of the infrastructure is that preference dictates that node 1 must be the first to be used. When its computing power is insufficient to process incoming messages, node 2 is activated, even though it has less NPC than node 3, as its preference is higher. The important thing about this new way of operating is that the administrator can now decide the exact order in which the nodes will be used, with their capacity being a secondary factor. If we look at the computing consumption, Fig. 16, we can see that nodes 2 and 3 use the same level of power, for example, at 9 a.m. However, the energy consumption, Fig. 17, at that same moment of node 2 is ten times lower than that of node 3.

In terms of infrastructure consumption over 24 h of use, as shown in Fig. 17, only the elements that are essential to cover the infrastructure's needs are active, without wasting available resources.

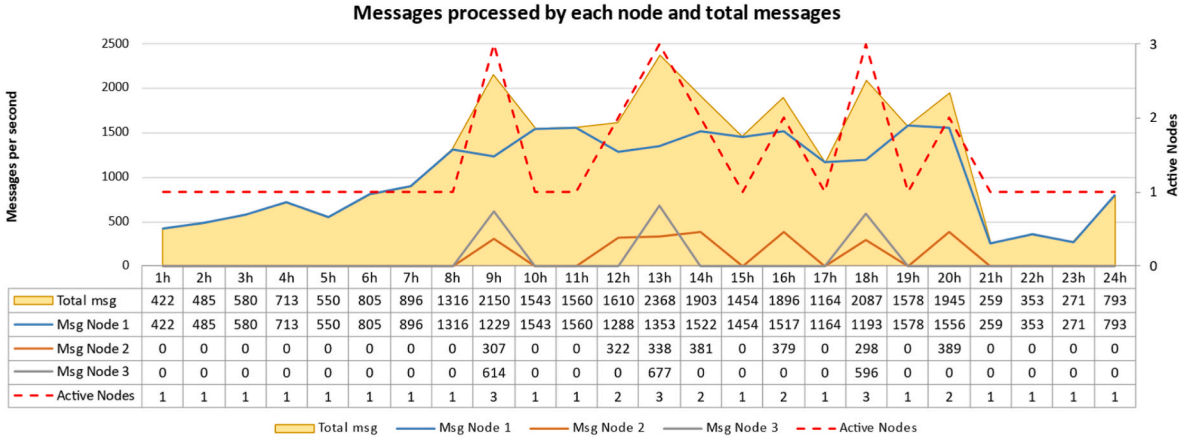
If we carry out a more exhaustive analysis of the improvement, Fig. 18 shows the consumption of the basic [14] infrastructure and the new proposal. It can be clearly seen (green bars) that the new architecture requires much less consumption per hour, except for peak hours of operation at specific times, but in no case does it get worse. In the initial situation, more than 3300 W were required in total operation of the three machines, this being an approximate value depending on the measurements obtained on a random working day. Under similar operating conditions and daily incoming messages, the new solution used about 1900 W per day. This implies a saving of more than 40 %.

In Fig. 19 we have expressed the consumption divided by the rate of messages per second that are processed in each hour. This allows us to obtain the average watts consumed per message/sec. Here it can be seen more clearly that our proposal manages to maintain a very constant cost between 0.05 and 0.10 W x msg/s, while the original solution consumed averages ranging between 0.10 and higher, even exceeding 0.35 W x 1000 msg/s. The golden line shows the percentage saving of our solution with respect to the initial one in terms of W x 1000 msg/sec consumed, which ranges between 50 % and 70 % savings. Only at peak processing times (9h, 13h and 18h) our proposal does not improve, but this is because all computational resources are at 100 % capacity.

7.2. Scenario B

One of the benefits sought with the new architecture is to enable system flexibility and scalability, so scenario A will be expanded to include several more University of Alicante campuses. Specifically, we will add the Alcoy campus and the La Nucia university centre. These campuses will be equipped with sensors related to weather conditions, presence, indoor environmental conditions, electricity consumption, water consumption, Wi-Fi network information and other various experimental sensors. These campuses provide a much smaller set of records, estimated at around 250,000 records for the Alcoy campus and 75,000 records for the La Nucia centre during peak hours. Each of these sites has its own processing node: Node 4 in Alcoy, consisting of an Intel Core i7-12700K, 16 GB RAM, 500 GB SSD; Node 5 at the La Nucia site, consisting of an Intel Core i3-2120, 8 GB RAM and 250 GB SSD. These nodes, which were previously dedicated exclusively to processing the sensor records of their headquarters, now form part of the resources of the new architecture, and the data generated by these headquarters will be processed together with that of the main headquarters, as if they were a single data flow. As in scenario A, the NPC of each of these nodes has been calculated using the same message dataset as before, obtaining an $NPC = 1100$ and $Preference = 0.1$ for Node 4, and an $NPC = 800$ and $Preference = 0.8$, leaving the NPC/Preference configuration as shown in Table 5.

To test the scenario, the performance of both the original system and the new architecture deployment was measured over a 24-h period, and



	1h	2h	3h	4h	5h	6h	7h	8h	9h	10h	11h	12h	13h	14h	15h	16h	17h	18h	19h	20h	21h	22h	23h	24h
Total msg	422	485	580	713	550	805	896	1316	2150	1543	1560	1610	2368	1903	1454	1896	1164	2087	1578	1945	259	353	271	793
Msg Node 1	422	485	580	713	550	805	896	1316	1229	1543	1560	1288	1353	1522	1454	1517	1164	1193	1578	1556	259	353	271	793
Msg Node 2	0	0	0	0	0	0	0	0	307	0	0	322	338	381	0	379	0	298	0	389	0	0	0	0
Msg Node 3	0	0	0	0	0	0	0	0	614	0	0	0	677	0	0	0	0	596	0	0	0	0	0	0
Active Nodes	1	1	1	1	1	1	1	1	3	1	1	2	3	2	1	2	1	3	1	2	1	1	1	1

Fig. 15. Active nodes, total messages received and messages processed by each node.

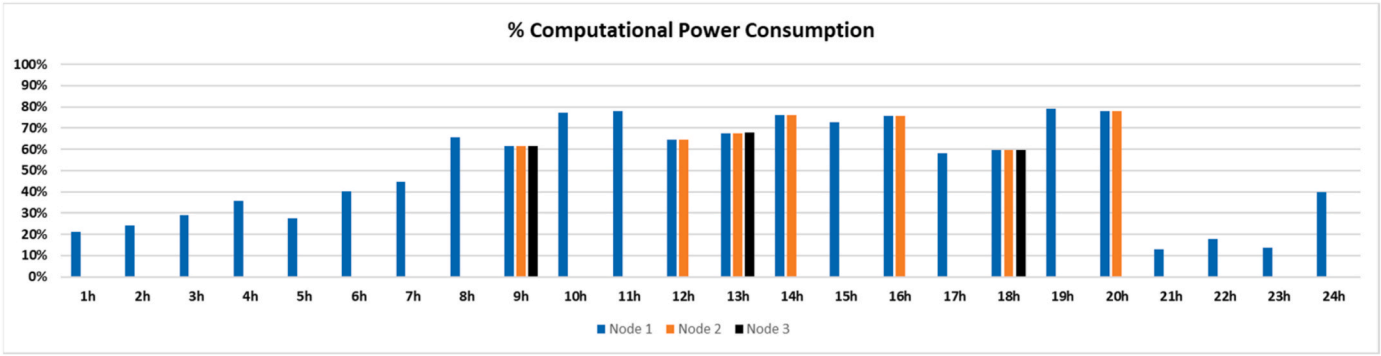


Fig. 16. Power measurement used for processing.

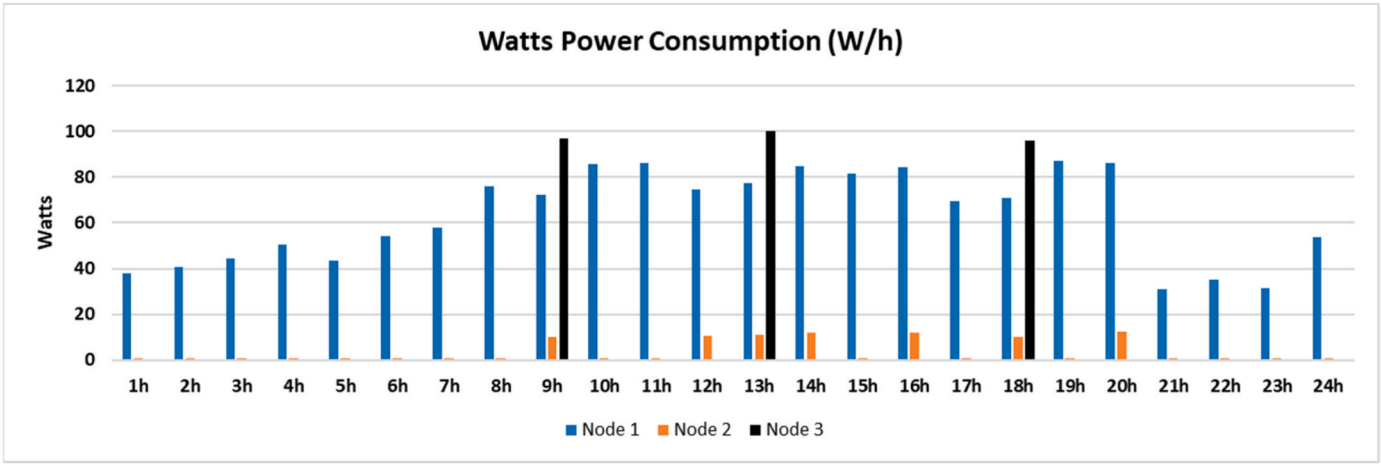


Fig. 17. Efficient Watts power consumption per hour of use.

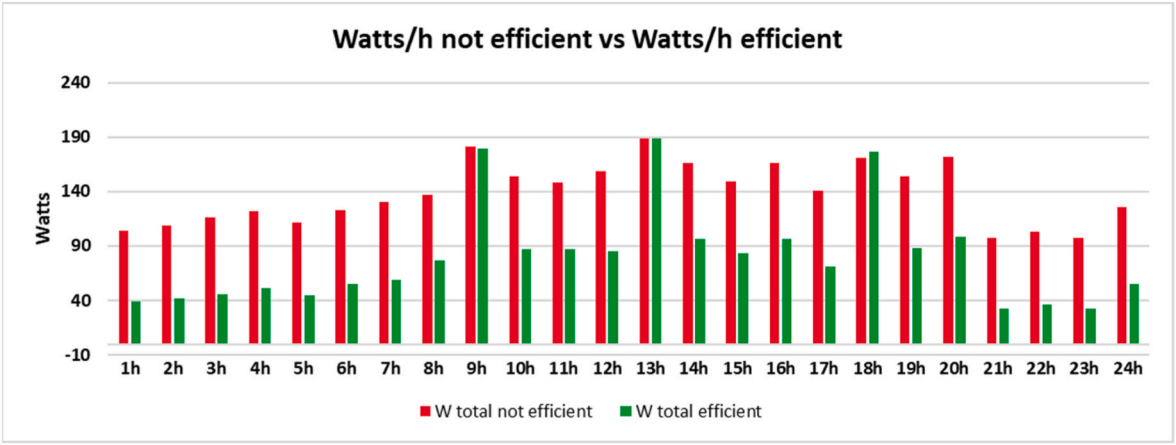


Fig. 18. Comparison of inefficient vs. efficient hourly consumption.

the following data was obtained. It should be noted that, in the original system, the five processing nodes are always on, while in the new architecture, when a node is not needed, it remains on standby. Fig. 20 shows the operation of the original system, where all nodes are processing messages. It can be seen that all five nodes are always active and that, during the middle of the day, between 8:00 a.m. and 9:00 p.m., system activity is highest.

This activity means that each node must always be processing messages and therefore consuming energy. Fig. 21a shows the computing power consumed by each node and Fig. 21b shows the energy in watts consumed based on the power used. As can be seen, all nodes are running, which means that they all consume a lot of energy.

When applying our architecture, you can see how computational resources are quickly optimised. Fig. 22 shows how the scenario works, using the same message load to process, but this time using the new architecture. The first thing to note is that now, when processing the same load, it is common for several nodes to be inactive and therefore consume only standby power. In fact, in general, two or three nodes are more than enough to process the entire workload, although the system would have enough power to handle much more load.

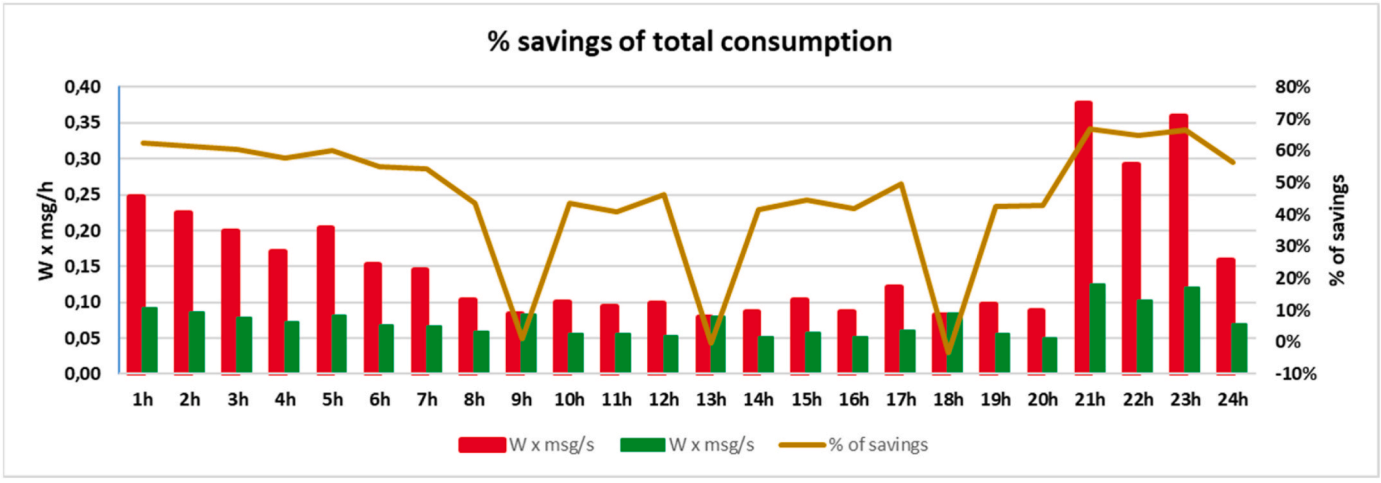


Fig. 19. Comparison of % savings in consumption.

Table 5
Configuración de los NPC y Preferencias de los nodos de computación del Escenario B.

Feature	San Vicente			Alcoy	La Nucia
Node	Node 1	Node 2	Node 3	Node 4	Node 5
NPC	2000	500	1000	1100	800
Preference	1	0.5	0.2	0.1	0.8

The effect of this operation has a rapid impact on energy consumption. Fig. 23a now shows the power consumed by each node to process messages. In this case, it is clear that only some nodes are working. This is because during off-peak hours, the power of Node 1 is practically sufficient, while during the rest of the day, Nodes 1, 5 and 3 can sometimes process the entire message flow. It should be remembered that according to the preference (Table 5), the order of activation of the nodes is: 1, 5, 2, 3, 4. Likewise, as shown in Fig. 23b, the consumption of each processor is lower when a node is not processing data, as it only consumes standby power. Furthermore, as the computational load is distributed evenly among all nodes (Fig. 23a), the nodes tend to distribute their consumption among all of them.

When observing the consumption of the new architecture, savings of up to 65 % in wattage consumption can be seen. This is because the new architecture gives priority to the most efficient nodes, meaning that only those that consume the least are used, and only when absolutely necessary.

Finally, if we look at the comparison between the consumption of both modes of operation, traditional inefficient architecture and efficient architecture, we can see that there is a big difference in the energy consumed. Fig. 24 shows the consumption of the traditional system (red) compared to the consumption of the efficient system (green). It should be noted that the same number of messages are being processed, and that the traditional system, which requires all its nodes to be always switched on, generates a minimum consumption that is not far from the consumption during peak hours. However, the efficient architecture consumes much less energy, especially during off-peak hours, as it keeps the nodes inactive. And if we compare again the energy cost of processing 1000 msg/h, Fig. 24b, we observe two effects. First, there is again a significant saving as the energy cost of processing is reduced, and second, and more importantly, the energy cost stabilises again. The new architecture shows that it maintains consumption within a range of 0.05 to 0.10 W, which demonstrates that, as the load increases, it is possible to keep the cost under control.

As mentioned above, the golden line in Fig. 24b shows the hourly

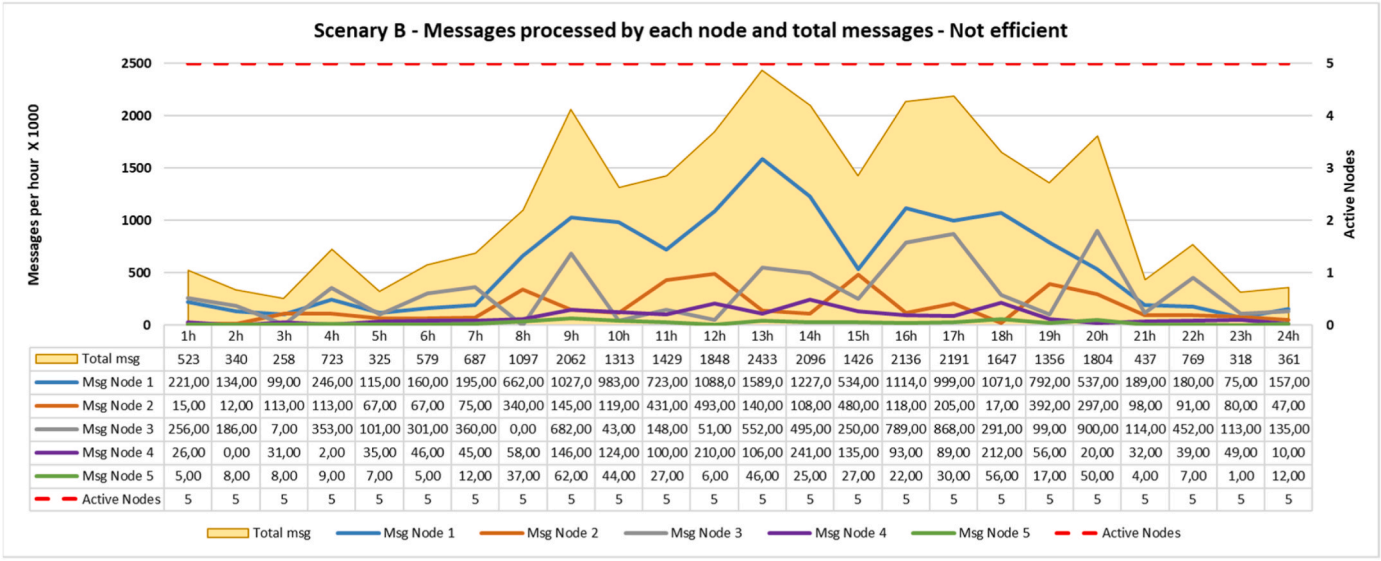


Fig. 20. Messages processed by each node and total messages processed.

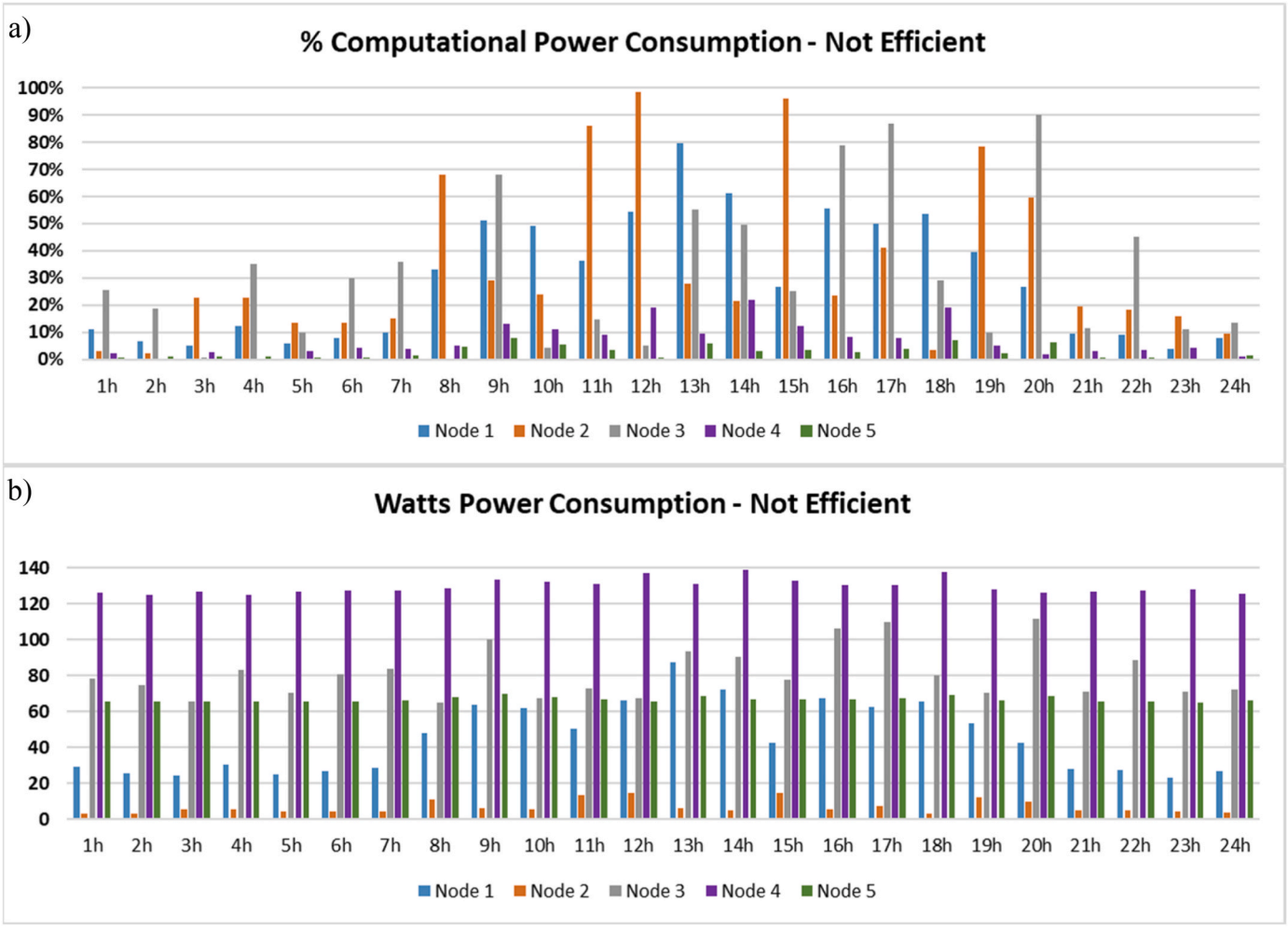
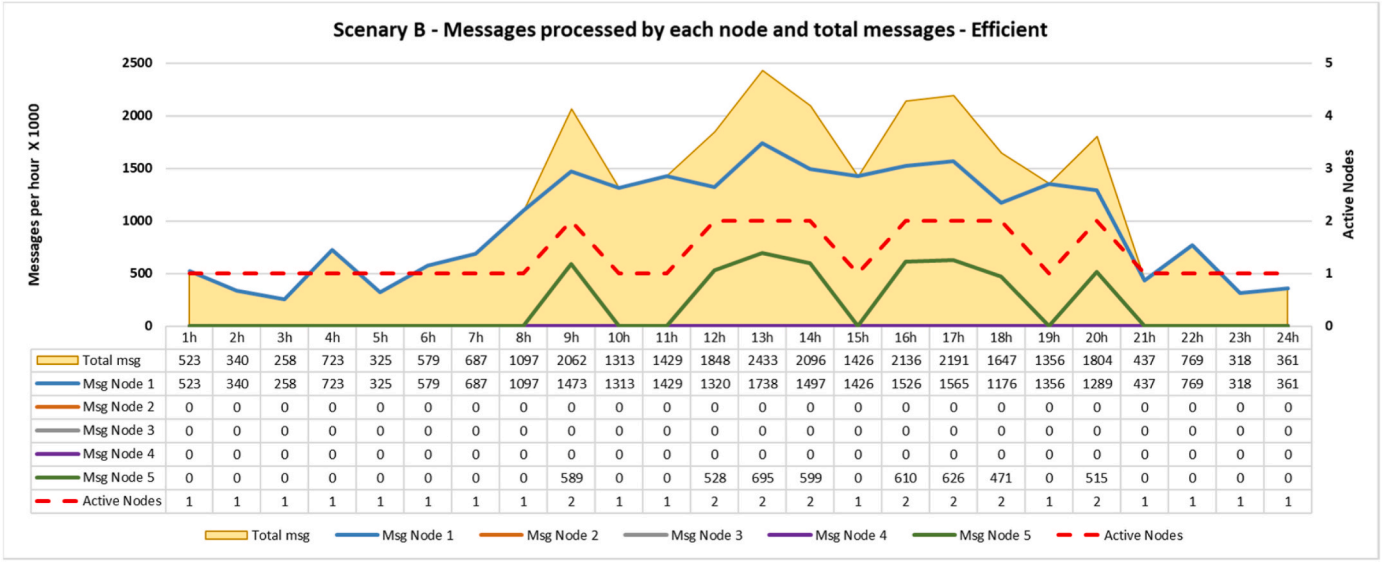


Fig. 21. a) Computational capacity consumed by the processor to process messages. b) Energy consumption of each processor based on the messages processed.



	1h	2h	3h	4h	5h	6h	7h	8h	9h	10h	11h	12h	13h	14h	15h	16h	17h	18h	19h	20h	21h	22h	23h	24h
Total msg	523	340	258	723	325	579	687	1097	2062	1313	1429	1848	2433	2096	1426	2136	2191	1647	1356	1804	437	769	318	361
Msg Node 1	523	340	258	723	325	579	687	1097	1473	1313	1429	1320	1738	1497	1426	1526	1565	1176	1356	1289	437	769	318	361
Msg Node 2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Msg Node 3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Msg Node 4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Msg Node 5	0	0	0	0	0	0	0	0	589	0	0	528	695	599	0	610	626	471	0	515	0	0	0	0
Active Nodes	1	1	1	1	1	1	1	1	2	1	1	2	2	2	1	2	2	2	1	2	1	1	1	1

Fig. 22. Messages processed by each node, number of active nodes and total messages processed using an efficient architecture.

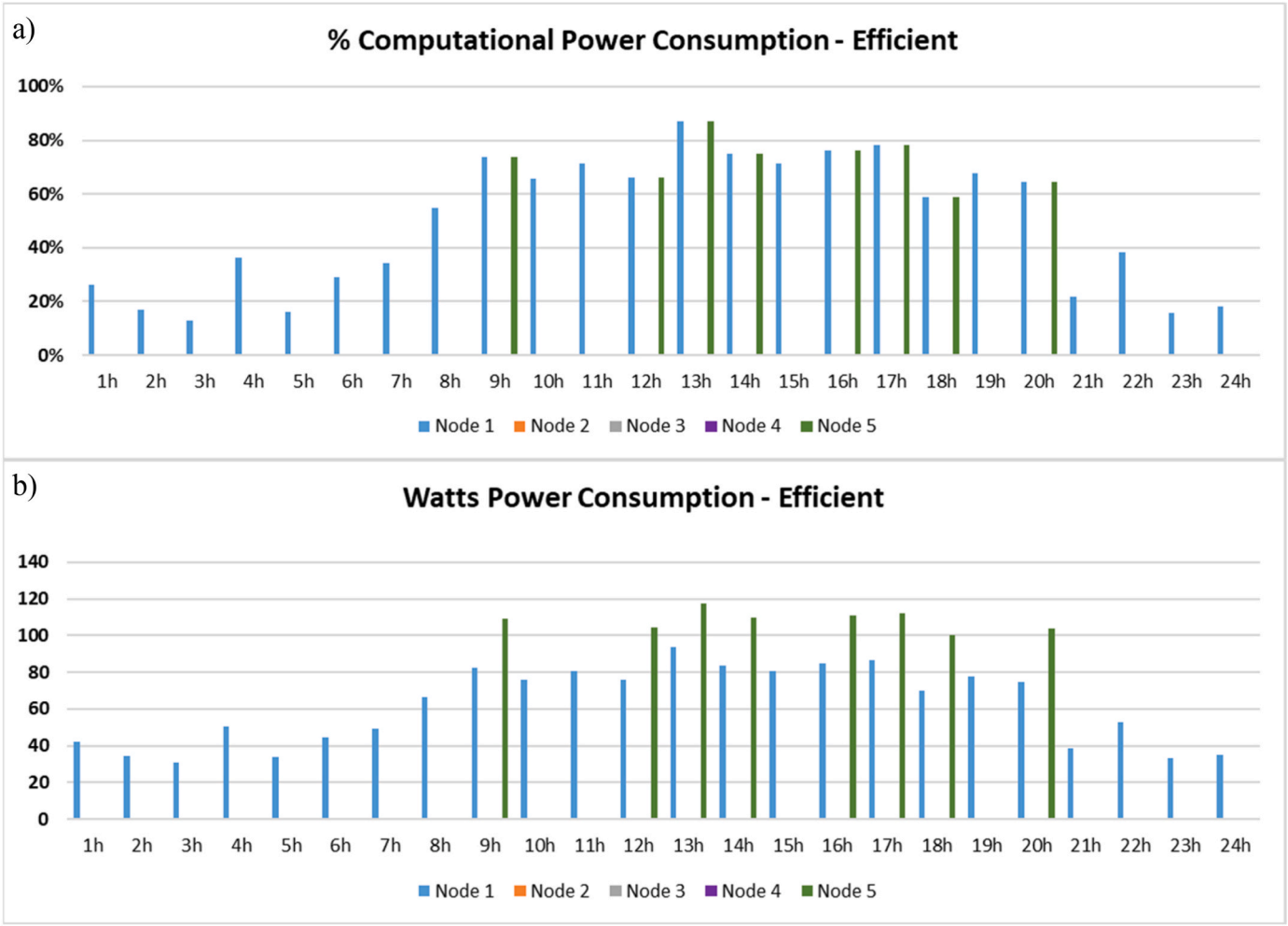


Fig. 23. a) Computational capacity consumed by the processor to process messages. b) Energy consumption of each processor based on the messages processed.

watt savings consumed by the nodes, which remains between 40 % and 90 %, giving an average saving of more than 65 %.

7.3. Stability study

To verify the long-term stability of the new architecture, a study was conducted on the power consumed over 30 days, controlling processing using scenario B with five nodes. For the study, the infrastructure was operated using both options, the traditional one and the efficient architecture, and the average daily consumption per 1,000 msg/h was finally calculated. The results are shown in Fig. 25. During the monitoring period, it can be seen that on Saturdays and Sundays, the number of messages to be processed was lower, around 30 % of weekdays, but even when the architecture operates with few messages, consumption continues to improve. Fig. 25 shows the average daily consumption of the traditional system in red and the average daily consumption of the efficient architecture in green. The gold line shows the consumption savings of the new system. It can be seen that only when there are very few messages to process are both costs similar, but even on those days, improvements are achieved. Therefore, based on the experiment, it can be considered that the new proposal generates a more efficient stable system.

7.4. Analysis of traditional solutions on scenarios

In the previous sections, the performance of the new architecture has been experimentally compared with the original system [36]. The provisioning mode of the original system was based on resource reservation, the PPPA technique. As explained in both scenarios, there were several processing nodes, each specialised in processing messages from each campus. Each node, being dedicated to a specific message flow, ensures that these messages will be processed, but generates a waste of resources when they are not being used. This is the typical behaviour of the PPPA strategy, where a resource is allocated to a task based on the expected prediction, which means that when that prediction is lower, the resource is wasted. This is reflected in the calculations in Figs. 18 and 24-a, where it can be seen that the dynamic allocation strategy is much more efficient.

As for ODPPA, this strategy dynamically allocates resources but seeks the nearest node with sufficient resources. This strategy is not the most effective in our scenarios, as demonstrated by Scenario B. In this scenario, the La Nucia centre has a processing node, but this node is one of the most expensive. That is why, although Node 5 is the closest node for processing the messages produced on this campus, priority is given to other more suitable nodes to process these messages, even if they are not the closest. This strategy of searching for the most suitable processing node rather than the closest one is achieved by using the *Preference* and *NPC* of the nodes explained above.

Regarding strategies such as ODPSBA, in our scenarios where environments contain multiple sensors that generate messages related to security aspects such as fire alarms, intrusion into spaces, etc., it is necessary to use an instant resource allocation strategy. This means that it is not possible to wait for a complete batch of messages, since if an alarm message arrives, it must be processed immediately, ensuring a

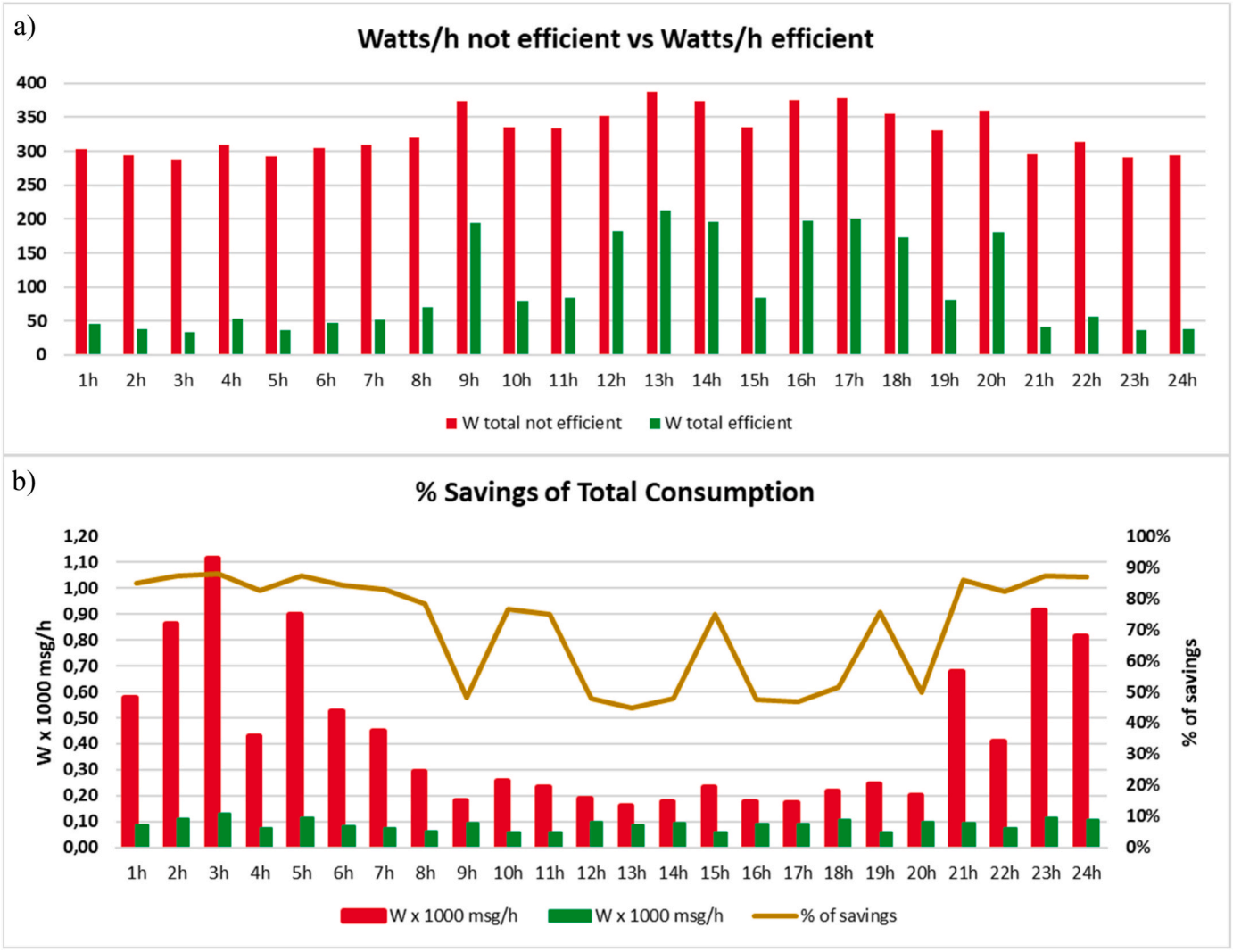


Fig. 24. a) Total watts consumed by traditional architecture (red) versus watts consumed by efficient architecture (green). b) Consumption in watts per 1,000 messages per hour of the traditional system (red) and the efficient architecture (green), and the % savings in watts of our architecture compared to the original.

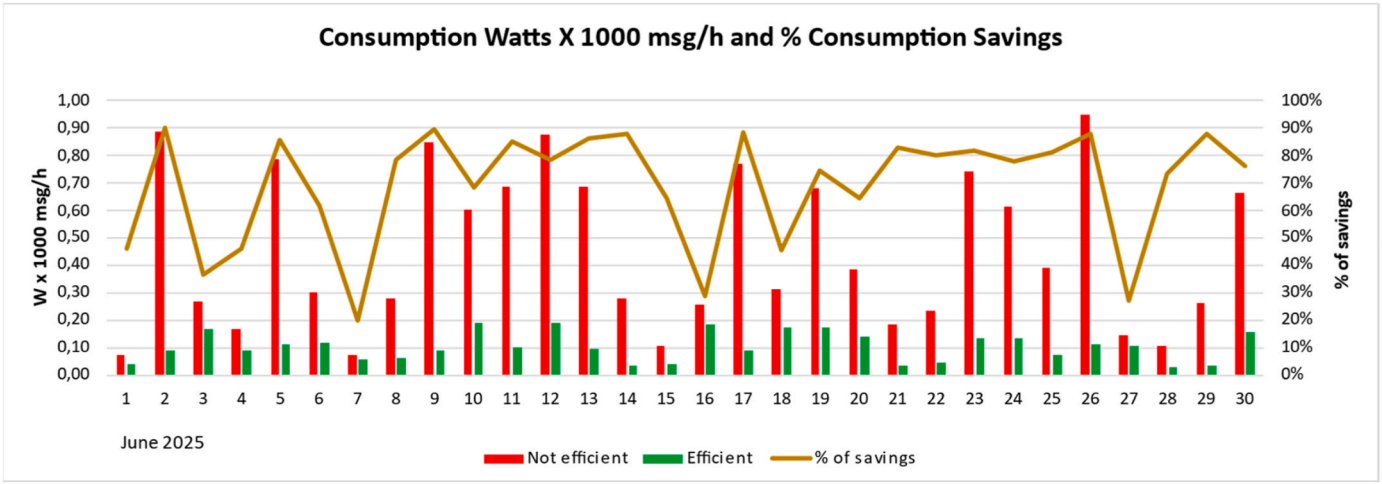


Fig. 25. Consumption in watts per 1000 messages per hour of the traditional system (red) and the efficient architecture (green), and the % watt savings of one architecture versus the other in June 2025.

resource. For all these reasons, ODPSBA strategies are not suitable for systems that require a rapid response to an event.

8. Conclusions

The FOG computing paradigm is essential for Internet of Things (IoT) environments, as it enables the decentralisation of data processing and the optimisation of resources. However, efficient management of this intermediate layer is hampered by the dynamic variability of computational demand in large-scale deployments, revealing inefficiencies in traditional architectures that rely on manual management or static allocation. Furthermore, because it is FOG, node availability can be compromised, so any architecture must provide support for fault management.

This work addressed these issues by designing and validating an advanced, dynamic, self-scalable, and secure FOG layer. The proposal introduces a structured architecture that implements control automation and scaling through an advanced control layer, consisting of a coordinator node and a balancer node. The central component of this solution is the Simple FOG Management Protocol (SFMP), developed specifically to facilitate automated resource management, ensure efficient allocation of computational capacity, and provide robust fault tolerance mechanisms. The experimental results and analysis of the architecture reveal several key conclusions.

The proposal optimises resource use and saves energy. Implementing the architecture in complex real-world scenarios on the University of Alicante campus demonstrated a substantial optimisation in resource use. By using a selective activation strategy based on processing demand, preference, and capacity thresholds (80 %), the system avoids computational waste. This resulted in a significant reduction in energy consumption, with savings ranging from 40 % to 65 % compared to traditional allocation approaches.

Prioritisation has been incorporated through the evolution of the Coordinator's load allocation algorithm, introducing the Preference parameter. This allows administrators to decide the exact order of use of the nodes, which is crucial for efficiency, as it makes it possible to prioritise nodes with lower operating costs or better energy efficiency, even if they are not the most powerful, thus maximising savings.

The SFMP protocol and expanded architecture incorporate backup elements (backup coordinator and balancer) and error control messages ("ARE YOU ALIVE?") for detecting and managing potential infrastructure failures. This ensures system availability by automatically reconfiguring the load in the event of a failure. The activation of a replacement element occurs in a matter of seconds, without a significant impact on data processing flow. The impact of the SFMP protocol on infrastructure communications is practically negligible. In an environment that handles millions of messages per hour, the number of control and management messages (including responses) is completely acceptable and does not affect system performance.

In summary, dynamic, self-scaling FOG architecture that is aware of administrator preferences, together with the SFMP protocol, provides a robust, adaptable and energy-efficient solution for managing complex and variable IoT environments.

As a line of future work, several avenues for further improving the efficiency of the FOG layer have been identified. Since the current architecture already provides continuous workload monitoring, which is essential for dynamic control, the next stage will focus on integrating predictive capabilities. On the one hand, integrating predictive algorithms to forecast future workloads, which will facilitate early activation of nodes and allow for the complete disconnection of resources when their inactivity is predictable. And on the other hand, dynamic threshold modulation, since currently, the coordinator uses a static threshold (e.g. 80 % of NPC) to activate a new node. Predictive capabilities will allow this threshold to be dynamically adjusted, avoiding unnecessary activation of resources if the load is predicted not to exceed the combined capacity of the active nodes, thus maximising efficiency without compromising service availability.

CRediT authorship contribution statement

Lucia Arnau Muñoz: Writing – original draft, Methodology, Investigation, Formal analysis, Conceptualization. **José Vicente Berná Martínez:** Writing – review & editing, Validation, Supervision, Funding acquisition, Conceptualization. **David Saavedra Pastor:** Writing – review & editing, Validation, Data curation. **Iren Lorenzo Fonseca:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work has been funded by the PID2023-152566OB-I00 project "Preventive maintenance of digital infrastructures through the application of artificial intelligence for the diagnosis and prediction of anomalies" financed by MICIU/AEI/10.13039/501100011033 and FEDER, UE.

Data availability

No data was used for the research described in the article.

References

[1] Elijah O, Rahman TA, Orikumhi I, Leow CY, Hindia MN. An overview of internet of things (IoT) and data analytics in agriculture: benefits and challenges. IEEE Internet Things J 2018;5(5):3758–73.
[2] Trayush T, Bathla R, Saini S, Shukla VK. Iot in healthcare: challenges, benefits, applications, and opportunities. In: 2021 international conference on advance computing and innovative technologies in engineering (ICACITE). IEEE; 2021, March. p. 107–11.
[3] Chowdhury A, A Raut S. Benefits, challenges, and opportunities in adoption of industrial IoT. International Journal of Computational Intelligence & IoT 2019;2 (4).
[4] Lawal K, Rafsanjani HN. Trends, benefits, risks, and challenges of IoT implementation in residential and commercial buildings. Energy and Built Environment 2022;3(3):251–66.
[5] Macia Pérez F, Berna Martínez JV, Lorenzo Fonseca I. Modelling and implementing smart universities: an it conceptual framework. Sustainability 2021;13(6):3397.
[6] Sriram GS. Cloud computing: an overview of big data challenges and opportunities for large enterprises. Int Res J Mod Eng Technol Sci 2022;4(1):1331–7.
[7] Saad HM, Shdefat A, Nawaz A, El-Sherbeeny AM, El-Meligy MA, Rana MRRR. Enhancing energy balance in wireless sensor networks through optimized minimum spanning tree. PeerJ Comput Sci 2024. https://doi.org/10.7717/peerj-cs.2269.
[8] Alsboui T, Qin Y, Hill R, Al-Aqrabi H. Distributed intelligence in the internet of things: challenges and opportunities. SN Computer Science 2021;2(4):277.
[9] Krishnamurthi R, Kumar A, Gopinathan D, Nayyar A, Qureshi B. An overview of IoT sensor data processing, fusion, and analysis techniques. Sensors 2020;20(21): 6076.
[10] Lin J, Yu W, Zhang N, Yang X, Zhang H, Zhao W. A survey on internet of things: architecture, enabling technologies, security and privacy, and applications. IEEE Internet Things J 2017;4(5):1125–42.
[11] Sethi P, Sarangi SR. Internet of things: architectures, protocols, and applications. Journal of electrical and computer engineering 2017;2017(1):9324035.
[12] Zare M, Sola YE, Hasanpour H. Towards distributed and autonomous IoT service placement in fog computing using asynchronous advantage actor-critic algorithm. Journal of King Saud University-Computer and Information Sciences 2023;35(1): 368–81.
[13] Bakar MTA, Jamal AA. Latency issues in internet of things: a review of literature and solution. Int J 2020;9(1.3).
[14] Muñoz LA, Martínez JVB, Asensi CC, Fonseca IL. Dynamic FOG computing for the efficient management of IoT infrastructures. En Lecture notes in networks and systems 2025:220–9. https://doi.org/10.1007/978-3-031-82073-1_22.
[15] Zahoor S, Mir RN. Resource management in pervasive internet of things: a survey. Journal of King Saud University-Computer and Information Sciences 2021;33(8): 921–35.
[16] Yao S, Li Z, Guan J, Liu Y. Stochastic cost minimization mechanism based on identifier network for IoT security. IEEE Internet Things J 2019;7(5):3923–34.

[17] da Rosa Righi R, Correa E, Gomes MM, da Costa CA. Enhancing performance of IoT applications with load prediction and cloud elasticity. Future Gener Comput Syst 2020;109:689–701.
[18] Skarlat O, Schulte S, Borkowski M, Leitner P. Resource provisioning for IoT services in the fog. Proc. IEEE 9th int. Conf. Service orient. Comput. Appl. (SOCA); 2016. p. 32–9.
[19] Giordano A, Spezzano G, Vinci A. Smart agents and fog computing for smart city applications. In: Smart cities: first international conference, Smart-CT 2016, Málaga, Spain, June 15-17, 2016, proceedings, 1. Springer International Publishing; 2016. p. 137–46.
[20] Etemadi M, Ghobaei-Arani M, Shahidinejad A. Resource provisioning for IoT services in the fog computing environment: an autonomic approach. Comput Commun 2020;161:109–31.
[21] Agarwal S, Yadav S, Yadav AK. An efficient architecture and algorithm for resource provisioning in fog computing. Int J Inf Eng Electron Bus 2016;8(1):48–61.
[22] Bittencourt LF, Diaz-Montes J, Buyya R, Rana OF, Parashar M. Mobility-aware application scheduling in fog computing. IEEE Cloud Comput Mar./Apr. 2017;4(2):26–35.
[23] Panda SK, Dhiman A, Bhuriya P. Efficient real-time task-based scheduling algorithms for iot-fog-cloud architecture. In: 2023 14th international conference on computing communication and networking technologies (ICCCNT). IEEE; 2023, July. p. 1–7.
[24] Adhikari M, Mukherjee M, Srirama SN. DPTO: a deadline and priority-aware task offloading in fog computing framework leveraging multilevel feedback queueing. IEEE Internet Things J 2019;7(7):5773–82.
[25] Mohan A, Kaseb AS, Lu YH, Hacker TJ. Adaptive resource management for analyzing video streams from globally distributed network cameras. IEEE Transactions on Cloud Computing 2018;9(1):40–53.
[26] Sami H, Mourad A. Dynamic on-demand fog formation offering on-the-fly IoT service deployment. IEEE Transactions on Network and Service Management 2020;17(2):1026–39.
[27] Ali M, Riaz N, Ashraf MI, Qaisar S, Naeem M. Joint cloudlet selection and latency minimization in fog networks. IEEE Trans Ind Informat Sep. 2018;14(9):4055–63.
[28] Intharawijitr K, Iida K, Koga H. Analysis of fog model considering computing and communication latency in 5G cellular networks. In: 2016 IEEE international conference on pervasive computing and communication workshops (PerCom workshops). IEEE; 2016, March. p. 1–4.
[29] Agarwal S, Yadav S, Yadav AK. An efficient architecture and algorithm for resource provisioning in fog computing. Int J Inf Eng Electron Bus 2016;8(1):48–61.
[30] Yousefpour A, Patil A, Ishigaki G, Kim I, Wang X, Cankaya HC, Jue JP. Qos-aware dynamic fog service provisioning. arXiv preprint arXiv:1802.00800 2018.
[31] Taneja M, Davy A. Resource aware placement of IoT application modules in fog-cloud computing paradigm. Proc IFIP/IEEE Symp Proc Netw Service Manag 2017:1222–8 (IM).
[32] Vilela PH, Rodrigues JJ, Solic P, Saleem K, Furtado V. Performance evaluation of a Fog-assisted IoT solution for e-Health applications. Future Gener Comput Syst 2019;97:379–86.
[33] Martinez I, Jarray A, Hafid AS. Scalable design and dimensioning of fog-computing infrastructure to support latency-sensitive IoT applications. IEEE Internet Things J 2020;7(6):5504–20.
[34] Arnau Muñoz L, Berna-Martinez JV, Maciá Pérez F, Lorenzo Fonseca I. Anomaly detection system for data quality assurance in IoT infrastructures based on machine learning. 2024.
[35] Zhang W, Zhang Z, Chao HC. Cooperative fog computing for dealing with big data in the internet of vehicles: architecture and hierarchical resource management. IEEE Commun Mag 2017;55(12):60–7.
[36] Munoz LA, Berná-Martínez JV, Asensi CC, Pastor DS. Design of a distributed and highly scalable fog architecture for heterogeneous IoT infrastructures. Int J Softw Eng Knowl Eng 2025;35(2):195–215.
[37] jvberna/simplefogmanagementprotocol: Simple FOG management protocol prototype applied to IoT infrastructures (n.d.). GitHub. https://github.com/jvberna/simplefogmanagementprotocol.